UNIVERSIDADE FEDERAL DE LAVRAS

Escola de Engenharia — Engenharia de Controle e Automação

GAT108 — Automação Avançada

PROJETO INTEGRADOR

Inteligência artificial de borda para sistemas ciberfísicos

https://github.com/admufla/AA2026_2

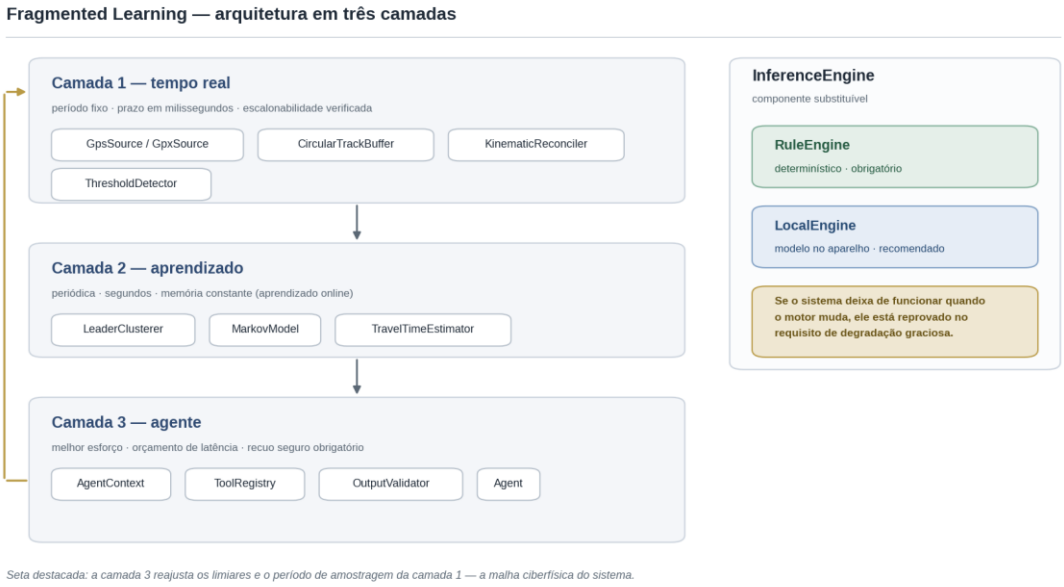


Figura 1 — arquitetura em três camadas do Fragmented Learning

Este documento é autossuficiente

Tudo o que você precisa está aqui ou nos links indicados. Nenhuma etapa depende de instrução adicional em sala. Leia por inteiro antes de escrever a primeira linha de código.

1. O problema real

Quem opera inteligência artificial hoje escolhe entre duas posições, quase sempre sem perceber que está escolhendo. Na primeira, a instituição envia os seus dados para um sistema que pertence a outro e recebe de volta uma resposta cujo funcionamento não pode auditar. Na segunda, ela retém o suficiente da cadeia para negociar os termos daquilo que não produz. A diferença entre as duas não é ideológica: é uma questão de quais camadas da produção de conhecimento a instituição controla, e é essa decomposição por camadas que de Miranda Neto (2026) propõe e que de Miranda Neto (2026b) leva à questão da soberania.

A segunda posição tem nome e tem limite. O nome é soberania de operação e de adaptação: controle sobre onde o sistema roda e sobre a última milha de conhecimento, língua e domínio. O limite é a soberania de gênese, que permanece vedada, porque treinar um modelo de fronteira consome capital que quase nenhuma instituição tem. A tese que organiza este projeto é que a primeira é alcançável hoje e vale a pena, e que a segunda não é, e que dizer isso com clareza vale mais do que prometer o contrário.

O Fragmented Learning, que vocês vão construir ao longo de dez semanas de construção, é a menor instância concreta dessa posição: um nó que adquire, corrige, aprende e decide sem enviar um único byte para fora. O Anexo G traz o mapa de leitura do paper de referência, a figura que mostra exatamente quais camadas este projeto toma e quais ele concede, e a regra de como discordar dele.

Feito o enquadramento, o problema concreto. Uma transportadora, uma prefeitura ou um produtor rural precisa saber onde estão seus ativos móveis e se algo saiu do previsto. Existem muitos negócios atualmente com problemas similares. A solução de mercado é um rastreador dedicado: hardware fechado, mensalidade por veículo, dados no servidor do fornecedor e um painel que mostra pontos em um mapa.

Esse arranjo tem três problemas de engenharia, e nenhum deles é sobre o mapa.

- O equipamento envia tudo para fora. Rastro de deslocamento é dado pessoal sensível sob a LGPD, e a decisão de exportá-lo raramente é técnica, é o que a arquitetura do fornecedor impõe.
- O sistema não aprende nada. Registra que o veículo está na rua X às 14h20, mas não sabe se isso é normal para uma terça-feira, porque nunca modelou o que é normal.
- O alerta é configurado à mão. Alguém desenha uma cerca virtual e define um limiar. Quando a operação muda, os limiares ficam obsoletos e ninguém percebe.

Enquanto isso, existe no bolso de todo mundo um dispositivo com receptor GNSS, unidade inercial, vários núcleos, memória de sobra e capacidade de executar um modelo de linguagem localmente. O hardware do problema já está resolvido. Falta engenharia de sistemas.

A pergunta da disciplina

Um nó de borda consegue aprender sozinho o envelope normal de operação de um ativo móvel, detectar desvios em tempo real e explicar o que observou, tudo no próprio aparelho, sem enviar um único byte para fora? E vários nós, cooperando entre si, conseguem fazer isso melhor que um nó sozinho?

Ao final do semestre a turma terá construído a resposta, mesmo que parcialmente, medido o desempenho dela e sabido dizer com onde ela falha. Essa última parte é a mais importante: sistemas de automação são julgados pelo comportamento nos limites, não pela demonstração feliz.

1.1. Por que este problema

Porque ele não deixa ninguém escapar de nenhum conteúdo da ementa – e em termos da avaliação é progressiva e dividida em múltiplas etapas; Aquisição periódica com prazo é tempo real. O buffer compartilhado entre a thread de aquisição, a interface e o agente é região crítica. Leituras ruidosas sob restrição cinemática são reconciliação de dados. A inferência disputando núcleos com as tarefas periódicas

é escalonabilidade. O aquecimento do aparelho realimentando a taxa de amostragem é sistema ciberfísico. A comunicação entre aparelhos é rede. E o dado é pessoal de verdade, o que torna a criptografia exigência, não exercício.

Há também uma vantagem prática: os dados são seus. Cada integrante grava o próprio deslocamento e a dupla trabalha com um conjunto real, não sintético.

2. O que cada dupla vai construir

Um sistema ciberfísico de borda: um aparelho que sente o mundo físico, corrige as próprias medições, aprende o comportamento normal daquilo que observa, decide, age e coopera com outros aparelhos, tudo sem nuvem. O paper de referência chama isso de instrumento de formação e o desenvolve no Apêndice C (de Miranda Neto, 2026b). O domínio do problema é escolha de cada dupla; a estrutura é a mesma para todas.

O que muda em relação a um assistente conversacional

Quatro coisas, e vale segurá-las desde já. O tempo vira restrição física, porque uma conversa pode esperar um segundo e um sistema que controla algo tem prazo fixo ou falhou. A parte durável do sistema é o agente, o código que sente, decide, verifica e age, enquanto o modelo de linguagem é ajudante opcional que pode ser trocado ou removido. O sistema mede quanto pode confiar em cada leitura de sensor antes de confiar em qualquer modelo. E vários aparelhos cooperam diretamente, sem nuvem no meio.

2.1. As três camadas e seus contratos temporais

Camada	O que faz	Contrato temporal
1 — Aquisição em tempo real	Sente, reconcilia leituras ruidosas e detecta eventos determinísticos	Período fixo, na ordem de milissegundos. Prazo verificado por cálculo e por medição, não afirmado.
2 — Aprendizado online	Aprende padrões recorrentes daquilo que observa	Periódica, na ordem de segundos, sob memória constante. Sem acumular histórico.
3 — Agente	Decide, valida, age e reconfigura a camada 1, fechando a malha	Melhor esforço, com orçamento de latência e recuo seguro obrigatório.

A regra que vale mais que todas as outras: o agente não é o modelo

O agente é código seu. O modelo de linguagem é peça substituível atrás de uma interface, e pode simplesmente não existir. O teste de aceitação é afiado: se o sistema para de funcionar quando o modelo é removido ou trocado, ele reprovou no requisito de degradação graciosa. Isso torna verificável, por um estudante que retira o modelo e observa o sistema continuar, a distinção entre soberania de operação e soberania de gênese que o paper desenvolve (de Miranda Neto, 2026b, seções 8.2 e C.2).

2.2. As dez capacidades exigidas de toda dupla ou indivíduo (as duas opções são válidas)

Estas capacidades não são negociáveis e não dependem do domínio escolhido. Elas derivam da Tabela C1 do paper de referência, que mapeia cada elemento do instrumento à competência que ele forma. O Anexo A traz cada uma com o critério de aceitação.



Figura 2 — as dez capacidades e a semana em que cada uma é entregue, iguais para todas as duplas

A leitura da figura é direta: todas as duplas ou indivíduos fazem tudo. Não há divisão de módulos entre duplas/indivíduos e não há dependência entre eles. O que diferencia uma dupla/indivíduo da outra/o é o problema que escolheu resolver, não o pedaço que lhe coube.

2.3. O caso inspirador

Este guia usa um caso inspirador, o Fragmented Learning, para explicar os problemas e dar à você um ponto de comparação: um nó de borda para monitoramento de ativo móvel, com aquisição de posição, reconciliação cinemática, aprendizado de lugares recorrentes, agente com motor substituível, persistência cifrada e frota multi-nó **(frota aqui representa multi-nó, assim a palavra frota é uma habilidade requerida, que você pode adequar a terminologia para seu problema)**. Ele não é fornecido como código e não é obrigação de domínio. Serve a três propósitos.

- Fixa o nível esperado. A especificação de contratos e testes do Anexo D define, semana a semana, o rigor do critério de aceitação. A sua dupla implementa os contratos e escreve os testes equivalentes no seu domínio.
- Serve de exemplo trabalhado. Quando o enunciado disser reconciliação sob restrição física, o caso inspirador mostra uma restrição física concreta e o que seria uma solução aceitável.
- É a referência de escopo mínimo. Uma dupla/indivíduo cuja proposta seja recusada duas vezes trabalha um escopo mínimo do próprio domínio, definido com o docente, sem prejuízo de nota nos demais critérios.

O que o caso inspirador NÃO é: código a copiar nem domínio a repetir. Não há repositório a clonar; cada dupla/indivíduo constrói o seu do zero. O Anexo C traz dez domínios inspiracionais, e pode ser proposto um décimo primeiro. O Anexo H descreve funcionalidades opcionais e não avaliadas que podem ser desenvolvidas por conta própria.

2.4. Requisitos não funcionais

Além das dez capacidades funcionais, todo sistema é avaliado por como se comporta sob restrição. Estes requisitos valem para qualquer domínio e cada um tem um critério verificável; a evidência entra no relatório e na apresentação.

Requisito	O que exige	Como se verifica
Desempenho	A aquisição cumpre o período; a inferência não estoura o orçamento de latência.	Latência de ciclo com média, medidos, não afirmados.
Escalonabilidade	Nenhum prazo perdido sob carga concorrente.	Tabela de tarefas com período, prazo e tempo de resposta; análise de utilização.
Consumo de energia	O aplicativo respeita um orçamento de energia por hora.	Medição de bateria em ensaio de trinta minutos; joules por ciclo estimados.
Aquecimento	A temperatura fica abaixo de um limite; o sistema recua a taxa ao aquecer.	O controle térmico reduz a taxa e o registro mostra a realimentação.
Proteção do aparelho	Treino e inferência respeitam limites de núcleos, bateria e temperatura.	Limites configuráveis; o escalonador pausa e retoma.
Segurança	Dados sensíveis cifrados em repouso; chaves em armazenamento seguro.	Cifra autenticada; chave no Keystore; nenhum dado pessoal em claro no disco.
Privacidade	Nenhum byte de dado pessoal sai do aparelho.	Inspeção de tráfego: zero envio com dado do usuário; conformidade com a LGPD.
Degradação graciosa	O sistema continua útil sem o modelo de linguagem.	Remover o modelo e observar o sistema seguir funcionando.
Uso de memória	O aprendizado opera em memória constante.	Memória constante ao longo do tempo, sem crescimento linear.
Confiabilidade	Falhas reproduzíveis e injetáveis.	Semente fixa; injeção de falhas com resultado determinístico.
Observabilidade	Estado e métricas visíveis localmente.	Registro e telemetria locais; painel de estado instrutivo.
Portabilidade	Roda no alvo mínimo, sem serviço pago.	minSdk declarado; funciona no emulador e em aparelho real.

Um requisito não funcional ausente, ou apenas afirmado sem medição, rebaixa o item correspondente ao nível Adequado.

3. Duplas/indivíduos independentes

Cada dupla/indivíduo escolhe o seu problema, constrói o seu sistema por inteiro e mantém o seu próprio repositório. Não há divisão de módulos, não há dependência e não há projeto comum: os projetos são independentes do começo ao fim.

O que é comum é o conjunto de exigências. Todos entregam as mesmas dez capacidades da seção 2.2, no mesmo nível, nas mesmas datas. É por isso que projetos diferentes podem ser avaliados pela mesma régua.

3.1. O repositório

Quem cria	**A dupla/indivíduo, na primeira semana, cria o próprio repositório do zero.**
Quem acessa	O docente, adicionado como colaborador ainda na primeira semana.
Contas admitidas	Somente contas do GitHub vinculadas ao endereço institucional @ufla.br, conforme a subseção 3.2.
Disciplina interna	Nada vai direto para a linha principal. Todo trabalho entra por pull request revisada pelo parceiro. Numa dupla isso parece cerimônia; é o que garante que os dois conhecem o sistema inteiro, que é o que a arguição cobra.
O que entra nele	Código, testes, os seus registros de sensor, docs/PROPOSAL.md, docs/DECISIONS.md e os relatórios. Nunca o arquivo do modelo.

3.2. Acesso: somente e-mail institucional

Configure o Git antes do primeiro commit

Commit feito com e-mail diferente do institucional NÃO é atribuído a você na apuração da contribuição individual, que vale oito pontos em cada avaliação. Corrigir autoria depois exige reescrever histórico, o que na prática não se faz. Faça isto antes da primeira linha.

```
git config user.email "seu.nome@ufla.br"

git config user.name "Seu Nome Completo"

git config user.email # confira

git log --format="%an <%ae>" | sort -u # confira a autoria
```

No GitHub, o endereço institucional precisa estar adicionado e verificado nas configurações da conta. Se você usa o recurso de e-mail privado, desative-o para este repositório. A verificação é objetiva e é feita antes de cada avaliação.

3.3. O que a dupla decide sozinha

Como os projetos são independentes, praticamente tudo é decisão dos responsáveis: o domínio, a linguagem, a tecnologia de interface, o modelo de concorrência, a persistência, o formato de serialização, o nível mínimo de interface de programação e a ambição quanto ao modelo local. A seção 6 traz cada uma dessas decisões com alternativas, prós e contras e uma recomendação do docente, que não é ordem.

O que a dupla/indivíduo NÃO decide está listado na subseção 6.11, e é curto: as dez capacidades, o padrão de evidência, as datas, a pontuação, a exigência de conta institucional, a execução local sem nuvem e a degradação graciosa.

4. A proposta de trabalho

Na segunda-feira, 31 de agosto de 2026, cada dupla/indivíduo apresenta à turma o problema que vai resolver. Esse é o momento em que o projeto deixa de ser exercício e passa a ser de vocês.

4.1. O que se espera de uma proposta

Não se espera originalidade. Espera-se adequação. Uma boa proposta é um problema real, pequeno o bastante para caber nas semanas previstas, cujo domínio comporta naturalmente as dez capacidades da seção 2.2. O critério de aprovação é esse e apenas esse.

O teste que separa uma proposta viável de uma inviável

Percorra as dez capacidades e responda, para cada uma, o que ela significa no seu domínio e qual será o teste de aceitação. Se em alguma delas a resposta for forçada, se não há grandeza física com incerteza declarada para reconciliar, se não há tarefa periódica com prazo, se não há decisão que o agente possa tomar, o domínio não serve, por mais interessante que o problema seja. É melhor descobrir isso cedo do que tarde. |

4.2. A ficha de proposta

Entregue **na segunda-feira, 31 de agosto de 2026**, comitada no repositório da dupla/indivíduo como docs/PROPOSAL.md. O modelo completo está no Anexo B. Ela tem cinco partes:

1. O problema, em cinco linhas: quem tem, por que dói, e o que hoje é feito na falta de solução.
2. O ativo ou o processo observado, e a grandeza física medida, com a fonte da incerteza. Sem isso não há camada 1.
3. A tabela de mapeamento das dez capacidades: o que cada uma significa no domínio, e qual será o teste de aceitação. É a parte que decide a aprovação.
4. Os dados: como serão obtidos, quantos dias, e como a separação entre treino e validação será congelada.
5. Os riscos: o que pode não funcionar, e qual é o plano B para cada risco.

4.3. A apresentação

Tempo	O quê	Como
10 min	**A proposta**	O problema, a grandeza medida e a tabela de mapeamento. Não gaste tempo com contexto: vá ao mapeamento, que é o que a turma precisa julgar.
5 min	**Arguição**	Perguntas da turma e do docente, dirigidas às capacidades do mapeamento.

4.4. O parecer

Aprovada	**A ficha aprovada passa a valer como contrato: o que foi declarado em cada capacidade é o que será cobrado nas avaliações.**
Aprovada com ressalva	Uma ou duas capacidades mal mapeadas. Correção da ficha até a semana seguinte, sem reapresentar.
A refazer	O domínio não comporta o conjunto. É necessário reapresentar. Persistindo a recusa, trabalha sobre o caso inspirador, sem perda de nota nos demais critérios.

A proposta compõe a nota da AV1, com peso de doze por cento, avaliada pela adequação do domínio e pela qualidade do mapeamento, não pela ambição do problema.

4.5. Mudar de ideia

Pode-se trocar de domínio mediante nova ficha e parecer, e isso não custa nota, contudo até um limite de duas semanas após a apresentação/aprovação. Depois disso, não pode: a reconciliação já está implementada sobre uma restrição física específica, e refazê-la consome o que resta do semestre.

5. Ferramentas: o que instalar e onde

Tudo abaixo é gratuito e não exige cartão de crédito, assinatura ou período de teste. Se alguma ferramenta pedir pagamento, você entrou no caminho errado — volte e confira o link.

Ferramenta	Para quê	Onde obter
Android Studio	Ambiente de desenvolvimento. Já inclui JDK e emulador.	developer.android.com/studio
Git	Controle de versão. O histórico é parte da avaliação.	git-scm.com/downloads
GitHub	O repositório. Conta gratuita basta.	github.com/signup
GPSLogger	Gravar trajetos reais em GPX. Código aberto.	gpslogger.app
AI Edge Gallery	Medir o desempenho de inferência do seu aparelho antes de programar.	github.com/google-ai-edge/gallery
LiteRT-LM	Execução de modelo de linguagem no aparelho (fase III).	ai.google.dev/edge/litert-lm/android
Modelos otimizados	Artefatos compactos prontos para borda.	huggingface.co/litert-community

5.1. Sem celular Android disponível

Não há problema. O emulador do Android Studio reproduz rotas GPX nativamente: controles estendidos, aba Location, sub-aba Routes, importar o arquivo. Todo o projeto funciona no emulador, com uma exceção: a execução do modelo de linguagem depende da arquitetura suportada pelas bibliotecas nativas. Se não estiver disponível, use o RuleEngine, registre o fato no relatório e siga. Isso não reduz a nota abaixo do nível Bom.

5.2. Nunca versione o modelo

O erro que mais custa caro no semestre

Arquivos de modelo têm centenas de megabytes. O GitHub recusa arquivos acima de 100 MB e, quando isso acontece, o histórico já está contaminado e limpar é trabalhoso. O .gitignore já exclui as extensões usuais.

```
adb shell mkdir -p /data/local/tmp/edgenode |

adb push modelo.litertlm /data/local/tmp/edgenode/ |

# qual arquitetura o seu alvo suporta?

adb shell getprop ro.product.cpu.abilist
```

6. As decisões que são suas

Esta é a seção mais longa do documento, e é assim de propósito. O projeto fixa as interfaces, os testes e os prazos; tudo o mais é escolha da dupla/indivíduo. Escolher dá trabalho, e a tentação de esperar o professor dizer qual é a resposta certa é grande, mas não existe resposta certa nestas questões, existe decisão justificada. Saber escolher entre alternativas viáveis, sob restrição de tempo e de competência da dupla/indivíduo, é exatamente o que se espera de um engenheiro de controle e automação.

O que esta seção faz e o que ela não faz

Ela apresenta cada decisão com as alternativas reais, os prós e contras de cada uma, as armadilhas conhecidas e uma recomendação explícita do docente. A recomendação não é ordem: é o que ele faria com as informações disponíveis. Se vocês decidirem diferente e justificarem no registro de decisões, não há perda de nota alguma. O que perde nota é decidir por omissão, descobrir numa semana tardia que ninguém decidiu nada e cada dupla/indivíduo fez de um jeito.

6.1. A conversa de decisão

Na terça-feira posterior à disponibilização deste documento, antes de escrever qualquer linha, dediquem-se e decidam. Não é reunião de turma: os projetos são independentes, e podem escolher linguagens diferentes sem nenhum prejuízo. O docente está presente para responder a dúvidas factuais, não para decidir.

Quem decide	**No caso de dupla, os dois integrantes, por acordo.**
O que sai	O arquivo docs/DECISIONS.md, comitado ainda na terça, com uma entrada por decisão no formato da subseção 6.10. Sem esse arquivo na linha principal, a semana 1 não está concluída.
Pode mudar depois?	Pode, e às vezes deve. Mudança exige nova entrada no registro, com a data, o motivo e o custo estimado. O que não se aceita é mudança silenciosa, porque o registro é o que explica ao docente por que o código está do jeito que está.

Uma orientação sobre o tom da conversa: a pergunta certa não é qual é a melhor tecnologia, é qual dessas alternativas conseguimos sustentar por dez semanas. Uma escolha tecnicamente superior que você não domina é uma escolha pior, porque a arguição cobra.

6.2. Decisão 1 — Linguagem: Java, Kotlin ou as duas

A decisão de maior impacto, e a única que precisa ser tomada, sem meio-termo. Numa dupla, o motivo é a arguição: se um integrante escreve em Kotlin e o outro só lê Java, a arguição vai expor isso, e a nota individual é de cada um.

	A favor	**Contra**
Java	Os contratos que você escrever em Java, inclusive os testes. Threads, executores e sincronização aparecem explícitos — o que é uma vantagem quando o assunto avaliado é justamente concorrência e tempo real.	ficam Verboso. A documentação oficial do Android é hoje majoritariamente escrita para Kotlin, e boa parte dos exemplos que vocês encontrarão precisará de tradução mental. O runtime de inferência local tem interface pensada para Kotlin.
Kotlin	É o idioma corrente do Android: documentação, amostras e respostas de fórum estão em Kotlin. Menos código para o mesmo resultado. Nulabilidade no sistema de tipos evita uma classe inteira de defeitos. Corrotinas simplificam muito o trecho assíncrono. Abre a porta para o Jetpack Compose. E o invólucro do modelo local encaixa naturalmente.	Corrotinas escondem as threads — e vocês vão precisar enxergá-las na análise de tempo real (ver 5.4).
As duas	Tecnicamente suportado sem esforço: os dois compilam para o mesmo bytecode, no mesmo módulo, e podem se chamar mutuamente. Permite manter os contratos em Java e escrever só o invólucro do modelo local em Kotlin, que é onde ele mais ajuda.	Duas convenções de estilo convivendo no mesmo repositório aumentam o atrito na revisão. Exige registro de decisões uma regra clara de quando cada linguagem é usada, sob pena de virar bagunça.

O que muda em cada camada

Camada	**Em Java**	**Em Kotlin**
Domínio (Fix, Place, Event)	Classes finais com campos privados e métodos de acesso. Verboso, mas explícito.	data class resolve em uma linha, com equals, hashCode e copy de brinde. Ganho claro.
Buffer e concorrência	synchronized, ReentrantLock e ExecutorService, tudo visível. Bom para o que se avalia aqui.	Mesmas primitivas disponíveis. Corrotinas são tentadoras, mas leia 5.4 antes.
Reconciliação e	Aritmética sobre arrays. Praticamente idêntico	Idêntico, com sintaxe um pouco mais

Camada	Em Java	Em Kotlin
aprendizado	nas duas.	curta para percorrer coleções.
Interface	Views com ViewBinding, que é o que o seu projeto já traz.	Views com ViewBinding ou Jetpack Compose. Ver 5.3.
Motor local de inferência	Exige um invólucro, porque a interface do runtime é pensada para Kotlin.	Encaixa direto. É aqui que o Kotlin mais compensa.

Armadilhas de convivência entre as duas linguagens

Kotlin chama Java sem cerimônia. O caminho inverso tem quatro pontos que quebram silenciosamente, e todos aparecem porque os testes fornecidos estão em Java:

1. Método estático. Os testes chamam FleetNode.estimateClockOffsetMs(...) e TelemetryFrame.parse(...) como estáticos. Em Kotlin, isso mora num companion object e só vira estático de verdade com a anotação @JvmStatic. Sem ela, o teste não compila.
2. Exceção verificada. Kotlin não tem exceção verificada. Se um método Kotlin lança uma exceção que o código Java precisa capturar explicitamente, anote com @Throws. O caso concreto é a interface Vault, cujos métodos declaram throws Exception.
3. Parâmetro com valor padrão. Java não enxerga valores padrão de Kotlin: ou anote com @JvmOverloads, ou escreva as sobrecargas na mão.
4. Nulabilidade. Tipos vindos do Java chegam ao Kotlin como tipos de plataforma, sem garantia de não nulo. O compilador deixa passar e o defeito aparece em execução. Ao implementar uma interface Java em Kotlin, seja explícito sobre o que pode ser nulo.

```
// Kotlin — o que os testes em Java esperam encontrar
class FleetNode(val nodeId: String) {
companion object {
@JvmStatic
fun estimateClockOffsetMs(t0: Long, t1: Long, t2: Long, t3: Long): Long { ... }
}
}
class AesGcmVault(key: ByteArray) : Vault {
@Throws(Exception::class)
override fun encrypt(plain: ByteArray): ByteArray { ... }
}
```

Como ligar o Kotlin no seu projeto

O projeto já traz o plugin declarado no catálogo de versões, desativado. Ligar é remover o comentário de uma linha e sincronizar. A partir daí, arquivos .kt e .java convivem no mesmo diretório de fontes, sem separação.

```
// app/build.gradle.kts

plugins {

alias(libs.plugins.android.application)

alias(libs.plugins.kotlin.android) // <- descomente esta linha

}
```

Recomendação do docente — Decisão 1 | | Se você já escreveram Kotlin antes: adotem Kotlin como | linguagem principal, com o seu projeto permanecendo em Java (não é preciso traduzi-lo — vocês o consomem). Se não: fiquem em Java como | principal e abram exceção apenas para o invólucro do modelo local, | que é onde o Kotlin resolve um problema real. Em qualquer dos casos, | NÃO traduzam o seu projeto: o tempo gasto nisso não aparece em nenhuma rubrica. | ==========+ **Uma observação honesta sobre assistentes de IA** | | Vocês vão usar assistentes, e isso é permitido. Vale saber que o | material recente de Android é predominantemente escrito em Kotlin, e | os assistentes tendem a produzir Kotlin mais idiomático e atual, | enquanto em Java às vezes devolvem padrões de uma década atrás — | AsyncTask, por exemplo, que está obsoleto. Isso é um argumento real a | favor do Kotlin. O contra-argumento também é real: os testes e o | projeto estão em Java, e quem não domina Kotlin terá mais | dificuldade de julgar se o que o assistente devolveu está certo. | Pesem os dois. |

6.3. Decisão 2 — Interface: Views ou Compose

Vocês precisam de cinco painéis: operação, eventos, aprendizado, desempenho e frota. Nenhum deles é sofisticado; o que eles precisam é atualizar com frequência sem travar e mostrar números legíveis na demonstração ao vivo.

	A favor	Contra
Views + ViewBinding	Layout em XML é fácil de revisar em pull request. Zero configuração adicional.	Mais código para listas que mudam sozinhas. Atualizar uma lista exige adaptador.
Jetpack Compose	Estado reativo cai como uma luva para painéis que mudam a cada segundo. Menos código para as listas de eventos e de nós. É o caminho atual do Android.	Exige Kotlin, mais o plugin do compilador e mais dependências.

Recomendação do docente — Decisão 2

Nenhuma rubrica pontua a escolha de tecnologia de interface; o que se avalia é se a informação necessária aparece na tela durante a demonstração.

6.4. Decisão 3 — Concorrência: threads e executores ou corrotinas

Esta decisão parece de estilo e não é. Ela toca diretamente no que a AV2 avalia, e vale ler com atenção mesmo que escolha Java.

A disciplina exige que vocês identifiquem as tarefas do sistema, meçam o tempo de execução de cada uma, atribuam período e prazo, calculem o tempo de resposta e concluam sobre escalonabilidade. Isso pressupõe saber quantas tarefas concorrem e sobre quantos processadores.

O ponto que costuma passar despercebido **Corrotinas não são threads. Muitas corrotinas se multiplexam sobre um pequeno conjunto de threads: o despachante padrão usa, por regra, tantas threads quantos forem os núcleos disponíveis, com o mínimo de dois. Isso significa que a sua "tarefa periódica" deixa de ter uma thread própria e passa a disputar um pool. A análise de Rate Monotonic continua possível, mas vocês precisam declarar explicitamente esse mapeamento no relatório: quais corrotinas rodam em qual despachante, qual o paralelismo dele, e como isso muda a interferência entre tarefas. Quem usar corrotinas sem discutir esse mapeamento perde pontos no critério de tempo real, e não no de concorrência.**		
	A favor	**Contra**
Thread e ExecutorService	Um para um entre tarefa e thread, o que torna a tabela de tarefas direta. É o vocabulário da bibliografia e do que se cobra na prova de escalonabilidade.	Mais código. Tempo limite exige Future com get e tempo, e o cancelamento é trabalhoso.
• *Corrotinas**	withTimeoutOrNull resolve o orçamento de latência em uma linha. Cancelamento estruturado. Muito menos código no laço do agente.	Exige Kotlin. Esconde as threads, o que obriga a um capítulo extra no relatório. Cancelamento é cooperativo: não interrompe chamada nativa bloqueada.

Vale registrar uma limitação que existe nas duas alternativas e que vocês vão encontrar na Fase II — parte 2: interromper uma inferência já em andamento não funciona de verdade. Nem Future.cancel com interrupção, nem o cancelamento de corrotina conseguem parar uma chamada nativa que está executando. O tempo limite faz o seu laço seguir adiante, mas a inferência continua consumindo processador até terminar. Reconhecer isso no relatório vale nota; fingir que o cancelamento funcionou custa nota.

Recomendação do docente — Decisão 3 **Usem Thread e ExecutorService na camada de tempo real, independentemente da linguagem escolhida, porque é ali que a correspondência entre tarefa e thread precisa ficar visível. Se a dupla escolher Kotlin, usar corrotinas na camada do agente é uma boa ideia: o orçamento de latência fica trivial e essa camada é de melhor esforço, sem análise de escalonabilidade. Misturar os dois modelos, cada um na camada onde é melhor, é uma decisão defensável e fácil de justificar no relatório.**

6.5. Decisão 4 — Persistência dos eventos cifrados

Alternativa	Quando compensa	O que custa
SQLiteOpenHelper	Zero dependência, controle total, funciona igual nas duas linguagens. Vocês escrevem o SQL e enxergam exatamente o que é gravado.	Verboso e sem verificação do SQL em tempo de compilação. Erro de digitação em nome de coluna só aparece rodando.
Room	SQL verificado na compilação, menos código repetitivo, consultas tipadas.	Processador de anotações no build, o que acrescenta configuração e tempo de compilação. Em Kotlin, exige KSP.
Arquivo cifrado em anexação	O mais simples de todos: um arquivo, uma linha por evento, tudo cifrado. Suficiente para o que a AV2 pede.	Sem consulta. Listar os últimos cinquenta eventos exige ler e decifrar tudo. Fica ruim quando o histórico cresce.

Recomendação do docente — Decisão 4

SQLiteOpenHelper com os campos sensíveis cifrados individualmente pelo AesGcmVault de vocês. Não usem biblioteca que cifre o banco inteiro: a rubrica avalia a sua implementação de AES-GCM, e delegar isso a uma biblioteca esvazia justamente o que está sendo medido. O Room é uma escolha legítima, mas resolve um problema que vocês não têm nesta escala.

6.6. Decisão 5 — Serialização

Aparecem dois problemas distintos de serialização no projeto, e a resposta certa não é necessariamente a mesma para os dois.

A chamada de ferramenta	**É o que o OutputValidator recebe do motor: um objeto JSON pequeno, possivelmente cercado de texto solto. Aqui não há razão para dependência nova. A biblioteca org.json já vem no Android e o seu projeto já a disponibiliza no classpath de teste. Funciona em Java e em Kotlin.**
O quadro de telemetria	Trafega entre aparelhos várias vezes por segundo, e cada byte é energia de rádio.

Se escolher Kotlin e quiser usar kotlinx.serialization no lugar do org.json para a chamada de ferramenta, é aceitável, mas note que ela exige mais um plugin no build e que o ganho aqui é pequeno, porque o objeto tem três campos. Bibliotecas baseadas em reflexão, como Gson, funcionam nas duas linguagens e não trazem vantagem nesta escala.

6.7. Decisão 6 — Vamos rodar o modelo no aparelho?

Esta decisão depende do hardware que tem em mãos, e por isso o levantamento inicial vem antes dela. O que a dupla decide é a ambição a que se compromete. O limite físico que a restringe tem nome na literatura, a parede de memória: aparelhos de borda carregam de quatro a doze gigabytes, o que basta para executar um modelo pequeno e não basta para adaptá-lo por inteiro (de Miranda Neto, 2026b, seção 6.2).

Obrigatório	**O RuleEngine, determinístico, para todas as duplas sem exceção. Ele é o piso de comportamento e a referência de comparação da AV2. Sem ele não há como afirmar que o modelo agregou coisa alguma.**
Recomendado	O LocalEngine em pelo menos um aparelho da dupla, para que a comparação exista e o experimento final tenha o que comparar. Não precisa ser em todos.
Aceitável	Se nenhum aparelho da dupla suportar o modelo. Nesse caso, registrem o levantamento de hardware no relatório, com a saída de adb shell getprop ro.product.cpu.abilist de cada aparelho, e concluam a AV2 comparando o RuleEngine consigo mesmo sob diferentes configurações. A nota máxima nessa hipótese é o nível Bom, e isso está explícito na rubrica.

6.8. Decisão 7 — Nível mínimo de API

O minSdk 26 corresponde ao Android 8.0. Baixar esse número aumenta o número de aparelhos compatíveis e tira recursos das suas mãos; subir faz o contrário.

Nível	Alcance	Consequência técnica
24	Praticamente qualquer aparelho em uso.	Perde a acurácia de velocidade informada pelo sensor, que é justamente o que alimenta a covariância da semana 2. Não compensa.
26	Ampla.	Tem a acurácia de velocidade e os nais de notificação. O tipo de serviço em primeiro plano precisa de verificação condicional em tempo de execução.
29 ou mais	Restringe alguns aparelhos antigos.	Simplifica o código do serviço em primeiro plano e da permissão de segundo plano, eliminando ramificações condicionais.

Recomendação do docente — Decisão 7

Mantenham 26 e só mudem se aparelho da dupla ficaria de fora. Trocar o nível mínimo no meio do semestre é barato; descobrir tardiamente que um integrante | não consegue instalar o aplicativo não é.

6.9. Decisão 8 — Como a dupla divide o trabalho

Decisão interna da dupla, mas com consequência direta na nota individual, e por isso está aqui.

Arranjo	Como funciona	Risco
Divisão por arquivo	Cada integrante é dono de um conjunto de classes e assina os commits correspondentes.	Um integrante nunca entende a metade do outro. Aparece na arguição.
Programação em par	Os dois no mesmo código, revezando quem digita, ou interage com a IA. Qualidade alta e conhecimento compartilhado.	A autoria dos commits fica concentrada em quem digitou. Use commits co-assinados, ou o registro de contribuição individual fica distorcido.
Alternado por semana	Um implementa, o outro revisa e escreve os testes; trocam na semana seguinte.	Exige disciplina para não virar divisão por arquivo disfarçada.

Recomendação do docente — Decisão 8

Alternado por semana, com quem revisa escrevendo os testes adicionais. É o arranjo que melhor equilibra as três coisas que a rubrica individual mede: contribuição atribuível, domínio do que o colega fez e capacidade de arguir. Se optarem por programação em par, aprendam a usar commits co-assinados na primeira semana — depois é tarde.

git commit -m "buffer: descarte do mais antigo ao exceder capacidade

Co-authored-by: Nome do Colega <email@ufla.br>"

6.10. O registro de decisões

Um arquivo, docs/DECISIONS.md, comitado na primeira semana e atualizado sempre que algo mudar. Uma entrada por decisão, no formato abaixo. É curto de propósito: cinco linhas por decisão bastam, e a seção 5 do relatório de cada avaliação faz referência a ele.

D-03 — Concorrência na camada de tempo real

- Data: 10/03/2026

- Decisão: Thread e ExecutorService na camada 1; corrotinas na camada 3.

- Alternativas consideradas: corrotinas em tudo; threads em tudo.

- Motivo: a tabela de tarefas da AV2 exige correspondência visível entre

tarefa e thread; a camada 3 é de melhor esforço e ganha com withTimeout.

- Consequências: o relatório da AV2 precisa mapear as corrotinas do agente

ao despachante e declarar o paralelismo dele.

- Decidido por: os dois integrantes, em 10/03.

Por que isso importa além da burocracia: quando alguém perguntar "por que não usamos XXX?", a resposta estará escrita, com data e motivo, e a discussão não recomeça do zero. Em projeto de dupla, decisão não registrada é decisão que será tomada de novo.

6.11. As decisões que não são suas

A lista é curta de propósito. Tudo o que não está aqui é da dupla/indivíduo.

- As dez capacidades do Anexo A e o padrão de evidência de cada uma. O domínio é seu; o que precisa ser demonstrado, não.
- As datas: proposta em 31 de agosto e 1º de setembro, AV1 (19 e 20 de outubro), AV2 (7 e 8 de dezembro), e as apresentações nas datas do rodízio.
- A pontuação das seções 9.2 e 9.3 e a regra de conversão de nível em pontos.
- A exigência de conta e endereço institucional para atribuição de autoria.
- A execução integralmente local, sem nuvem e sem serviço pago.
- A degradação graciosa: um sistema que para de funcionar quando o modelo é removido reprovou na capacidade 8, qualquer que seja o domínio.
- A proibição de versionar o arquivo do modelo.

6.12. Dúvidas frequentes na hora de decidir

Perdemos nota se escolhermos Java em vez de Kotlin?

Não, e nem o contrário. Nenhuma rubrica menciona linguagem. O que é avaliado é o comportamento do sistema, a evidência experimental e o domínio demonstrado na arguição. Escolha errada é a que a dupla não consegue sustentar, não a que está fora de moda.

No caso de duplas, um sabe Kotlin e outro não. O que fazemos?

Isso é argumento de peso para Java como principal, porque a nota individual mede domínio, não velocidade de digitação.

Uma dupla pode usar Kotlin enquanto outra usa Java?

Sim, sem nenhum problema: os projetos são independentes e nada exige uniformidade entre duplas. O que não funciona é dentro da mesma dupla, onde um integrante não lê o que o outro escreve.

Podemos mudar de linguagem no meio do semestre?

Podem. O custo é menor do que parece, porque as interfaces são fixas: você reescreve uma implementação por vez, e os testes continuam valendo. O que torna a migração cara é fazê-la sem registro e sem plano. Nova entrada no DECISIONS.md, com estimativa de custo, e sigam.

E se descobrirmos a frente que a decisão foi ruim?

Isso é resultado, não fracasso — e cabe na retrospectiva da Fase I, que existe exatamente para isso. Registrem o que aprenderam. Um relatório final que discute honestamente uma decisão técnica ruim e o custo dela vale mais que um que finge que tudo foi planejado.

Um resumo de uma linha

Decidam rápido, registrem por escrito, e sigam. Nenhuma dessas escolhas é irreversível, e todas são menos importantes que entregar o critério de aceitação da semana.

7. Contrato antes do código

Ninguém começa de uma página em branco. A especificação do Anexo D entrega os contratos e os testes exigidos, semana a semana. O ponto de partida correto é escrever primeiro as interfaces e os testes que falham, e só então implementar até que passem.

```
./gradlew test # tudo

./gradlew test --tests "*Week04*" # apenas a semana 4

# relatório em HTML:

app/build/reports/tests/testDebugUnitTest/index.html
```

Organize o seu repositório por semana e mantenha, desde o início, os testes que definem cada capacidade. A lista completa do semestre é a especificação do Anexo D, capacidade por capacidade.

7.1. Componentes de base: resolva cedo, mas resolva você

Componente	Por que resolvê-lo cedo
AcquisitionService	Serviço em primeiro plano, canal de notificação, tipo de serviço e ciclo de vida. É a maior fonte de travamento em projetos Android de graduação.
GpsSource	Conversão de Location para Fix, incluindo a leitura correta das acurácias informadas pelo sensor.
GpxReader	Interpretação do formato de intercâmbio de trajetos.
Matrix	Multiplicação, transposta e inversão por Gauss-Jordan. É sobre reconciliação, não sobre depurar álgebra linear.
Profiler	Instrumentação com média, percentis e máximo, para que a tabela de tarefas tenha números medidos.
NodeServer, NodeClient	Transporte HTTP mínimo entre aparelhos. O conteúdo da frota é deriva de relógio e reconciliação entre nós, não socket.
MainActivity	Fluxo de permissões na ordem correta. Pedir localização em segundo plano junto com a de primeiro plano é negado silenciosamente pelo Android.
pacote fakes	Implementação falsa de cada componente, para que nenhuma dupla fique bloqueada.

7.2. Uso de inteligência artificial

Permitido e incentivado. Estamos em Automação Avançada; seria incoerente proibir automação intelectual. A responsabilidade pelo resultado, porém, é integralmente sua, e há quatro verificações convergentes: histórico de commits com autoria institucional atribuída, arguição presencial nas apresentações, e execução dos testes sobre o código entregue.

Um conselho prático que não é retórica: assistentes de IA são muito mais úteis quando o pedido é delimitado. “Implemente o método latest(int n) desta interface, seguro para concorrência, satisfazendo estes testes” produz código utilizável. “Faça um app de GPS” produz quatrocentas linhas que não compilam. O método de contrato antes do código existe, em parte, exatamente para isso.

O relatório deve trazer uma declaração de uso de IA: quais ferramentas, em que partes, e o que você verificou por conta própria. Declarar não penaliza. Omitir, sim.

7.3. Sugestões de prompts

Como não há projeto pronto, a construção parte da especificação do Anexo D e de ferramentas de inteligência artificial, cujo uso é permitido e incentivado, declarado no relatório, com a responsabilidade pelo resultado permanecendo da dupla/indivíduo. Um bom prompt cita o contrato, pede o teste antes da implementação e exige a evidência de aceitação. Peça código que você consiga ler, rode os testes e não aceite o que não entende.

Um roteiro por capacidade, para adaptar ao seu domínio:

- Contratos: “Gere as interfaces em Java (ou Kotlin) para FixSource, Reconciler e Agent conforme estas responsabilidades [colar do Anexo D], com documentação e sem implementação; depois escreva os testes que falham para cada uma.”
- Reconciliação: “Implemente mínimos quadrados ponderados para [minha grandeza], com covariância vinda da acurácia do sensor, e um detector de erro grosseiro por resíduo normalizado; inclua três casos de teste com dados reais e o critério quantitativo de aceitação.”
- Concorrência: “Aponte a região crítica neste buffer compartilhado entre a aquisição e a interface e escreva um teste que rode a corrida dez vezes e falhe se houver condição de corrida.”

- Tempo real: “Escreva um serviço de aquisição periódica de período fixo que sobreviva ao aparelho bloqueado, e uma instrumentação que registre média, p95 e p99 do ciclo [1].”
- Aprendizado: “Implemente agrupamento incremental em memória constante e um preditor do próximo estado, com acurácia validada fora da amostra de treino.”
- Agente: “Escreva o laço do agente com uma ferramenta de abstenção e um motor de linguagem atrás de uma interface, de modo que o sistema continue funcionando se o motor for removido.”
- Segurança: “Implemente cifra autenticada em repouso com a chave no Keystore e mostre que nenhum dado pessoal fica em claro no disco.”
- Validação: “Monte um ensaio cego que separe ajuste de avaliação e um experimento que estime o ganho pela Lei de Amdahl; gere a tabela de métricas.”

O relatório declara quais prompts e ferramentas foram usados e o que a dupla verificou. Código gerado que a dupla não sabe explicar na arguição não pontua.

8. Calendário de dez semanas

A construção acontece em dez semanas, sempre às terças-feiras. Nas segundas há aula; nas terças, laboratório de execução. A proposta é apresentada antes do início da construção, em 31 de agosto e 1º de setembro. A AV1 fecha a semana 5 e a AV2 fecha a semana 10. Não são semanas de construção 13/10, 9 e 10/11 (Congresso UFLA) e 1/12; o calendário abaixo já as descarta.

Cada semana tem um entregável concreto e um critério de aceitação verificável, ligado ao item de pontuação que ele alimenta.

Semana	Terça	Foco	Entregável e critério
S1	15/09	Domínio, contratos e estruturas	Modelo de domínio e contratos no seu repositório, com testes que falham e invariantes validadas. (cap. 1)
S2	22/09	Reconciliação I: incerteza e acurácia	Covariância a partir da acurácia do sensor; buffer de leituras. (cap. 2)
S3	29/09	Reconciliação II e III: resíduo e erro grosseiro	Mínimos quadrados ponderados, resíduo reduzido em três janelas reais e erro grosseiro detectado. (cap. 2)
S4	06/10	Concorrência e aquisição periódica	Região crítica com dez execuções sem falha; aquisição periódica de trinta minutos com o aparelho bloqueado. (caps. 3 e 4)
S5	20/10	Ensaio reprodutível e **AV1**	Semente fixa, reprodução determinística e tabela de sensibilidade; relatório parcial e Apresentação I. (cap. 5)
S6	27/10	Escalonabilidade e instrumentação	Tabela de tarefas com período, prazo, jitter e tempo de resposta medido. (cap. 6)
S7	03/11	Aprendizado online e predição	Agrupamento incremental e predição validada em memória constante. (cap. 7)
S8	17/11	Agente com abstenção e modelo substituível	Laço de agente que decide, valida e abstém, e funciona com o modelo removido. (cap. 8)
S9	24/11	Criptografia e frota multi-nó	Cifra autenticada, chave no Keystore e cooperação entre nós.

[1] O **P95** e o **P99** de um ciclo (ou de uma requisição/latência) são percentis que medem o tempo de execução ou resposta. O **P95** indica que 95% dos ciclos terminaram em um tempo menor ou igual àquele valor, enquanto os 5% restantes demoraram mais. O **P99** indica que 99% dos ciclos terminaram nesse limite, deixando apenas o 1% mais lento na extremidade.

Semana	Terça	Foco	Entregável e critério
			(cap. 9)
S10	08/12	Validação cega, TRL e **AV2**	Protocolo cego, métricas, Lei de Amdahl e relatório final; Apresentação Final. (cap. 10)

As apresentações seguem o rodízio da seção 11: a Apresentação I (19/10 as duplas A, B e C; 20/10 as duplas D, E e F); a Apresentação Final (7/12 as duplas D, E e F; 8/12 as duplas A, B e C). O cronograma encontro a encontro está no Plano de Ensino 2026/2.

9. Como a nota é formada

Duas avaliações, cada uma valendo cem pontos. A nota final da disciplina é a média ponderada:

Nota final = 0,40 × AV1 + 0,60 × AV2

AV1: 100 pontos, entrega na terça da semana 5

AV2: 100 pontos, entrega na terça da semana 10

9.1. A regra de conversão de nível em pontos

Cada item é avaliado em um quatro de níveis, e cada nível vale uma fração fixa dos pontos do item. Não há nota entre níveis: isso torna o fechamento objetivo e permite que você calcule a sua própria nota antes de recebê-la.

Nível	Vale	Significa
Insuficiente	**0%**	Ausente, ou presente mas sem funcionar, ou sem a evidência exigida.
Adequado	**50%**	Funciona parcialmente, ou funciona sem a evidência quantitativa que o critério pede.
Bom	**75%**	Atende integralmente o critério de aceitação descrito no Anexo A.
Excelente	**100%**	Atende o critério e acrescenta a evidência ou a análise descrita na coluna Excelente do Anexo A.

Nível Bom é o alvo, não o teto rebaixado

Uma dupla que entrega tudo funcionando, com as evidências pedidas, e nada além, tira setenta e cinco por cento. Isso é uma nota boa e é o que se espera de um trabalho correto. Os vinte e cinco por cento restantes de cada item pagam a análise que vai além do funcionamento: a comparação, a discussão do limite, o caso de borda que ninguém pediu. Não confunda com fazer mais coisas; é fazer as mesmas coisas com mais rigor.

9.2. AV1 — cem pontos, entrega na semana 5

Item	Escopo	Pontos	Insuf.	Adeq.	Bom	Exc.	O que é avaliado
Proposta de trabalho	Dupla/Indivíduo	**12**	0	6	**9**	12	Apresentada antes da construção e aprovada. Avaliada pela adequação do domínio e pela qualidade do mapeamento das dez capacidades, não pela ambição do problema.
Capacidade 1 — Domínio, contratos e estruturas	Dupla/Indivíduo	**8**	0	4	**6**	8	Invariantes validadas, exceção específica, estruturas em memória constante.
Capacidade 2 — Incerteza e reconciliação	Dupla/Indivíduo	**20**	0	10	**15**	20	Covariância a partir da acurácia informada pelo sensor; resíduo reduzido em três janelas reais; erro grosseiro detectado.
Capacidade 3 — Concorrência e região crítica	Dupla/Indivíduo	**16**	0	8	**12**	16	Cópias defensivas; dez execuções do teste de corrida sem falha, registradas uma a uma.
Capacidade 4 — Aquisição periódica em tempo real	Dupla/Indivíduo	**8**	0	4	**6**	8	Trinta minutos contínuos com o aparelho bloqueado, demonstrado ao vivo.
Capacidade 5 — Ensaio reprodutível e injeção de falhas	Dupla/Indivíduo	**8**	0	4	**6**	8	Semente fixa, reprodução determinística e tabela de sensibilidade da detecção.
Relatório parcial	Dupla/Indivíduo	**8**	0	4	**6**	8	Relatório com as nove seções da subseção 10.2, entregue na semana 9.
Apresentação I	Individual	**12**	0	6	**9**	12	A apresentação da sua dupla na semana 5, com fundamentação, MVP ao vivo, resultados e arguição, pelos cinco critérios da subseção 11.4.
Contribuição individual verificada	Individual	**8**	0	4	**6**	8	Commits com autoria @ufla.br atribuída ao integrante, distribuídos ao longo das semanas, e pull requests revisadas pelo parceiro.
TOTAL		**100**					**Peso de 40% na nota final**

9.3. AV2 — cem pontos, entrega na semana 10

Item	Escopo	Pontos	Insuf.	Adeq.	Bom	Exc.	O que é avaliado
Capacidade 6 — Instrumentação e escalonabilidade	Dupla/Indivíduo	**12**	0	6	**9**	12	Ci medido com dispersão declarada, tempo de resposta calculado e conclusão explícita de escalonabilidade.
Capacidade 7 — Aprendizado online e predição	Dupla/Indivíduo	**16**	0	8	**12**	16	Padrão recorrente aprendido sob memória constante e predição validada em dados retidos.
Capacidade 8 — Agente, abstenção e modelo substituível	Dupla/Indivíduo	**20**	0	10	**15**	20	Orçamento respeitado, saída validada, abstenção acionada, e o sistema seguindo com o modelo removido em execução.
Capacidade 9 — Criptografia e frota multi-nó	Dupla/Indivíduo	**12**	0	6	**9**	12	IV variável, chave no armazenamento seguro, modelo de ameaça escrito; dois ou mais nós, deriva medida, evento fundido.
Capacidade 10 — Validação cega, TRL e Amdahl	Dupla/Indivíduo	**12**	0	6	**9**	12	Quatro métricas nos dois motores, speedup com cinco repetições e dispersão, TRL justificado nível a nível.
Relatório final e leitura em camadas	Dupla/Indivíduo	**8**	0	4	**6**	8	Relatório consolidado com a seção de leitura em camadas exigida na subseção 10.2.
Apresentação Final	Individual	**12**	0	6	**9**	12	A apresentação da sua dupla na semana 15, pelos mesmos cinco critérios da subseção 11.4.
Contribuição individual verificada	Individual	**8**	0	4	**6**	8	Commits com autoria @ufla.br atribuída ao integrante nesta fase, e pull requests revisadas pelo parceiro.
TOTAL		**100**					**Peso de 60% na nota final**

9.4. Um exemplo de fechamento

Uma dupla/indivíduo entrega tudo funcionando, com as evidências pedidas, e sem análise adicional: todos os itens de dupla saem no nível Bom. Na apresentação, no caso de uma dupla, um integrante conduziu a fundamentação e respondeu bem, ficando no nível Excelente; o outro apresentou apenas o MVP e respondeu com hesitação, ficando no Adequado. A contribuição dos dois é Bom.

AV1 itens de dupla (80 pontos disponíveis) no nível Bom 60,0

integrante 1: apresentação Excelente (12) + contribuição Bom (6) .. 18,0 -> 78,0

integrante 2: apresentação Adequado (6) + contribuição Bom (6) .. 12,0 -> 72,0

AV2 itens de dupla (80 pontos disponíveis) no nível Bom 60,0

ambos: apresentação Bom (9) + contribuição Bom (6) ..... 15,0 -> 75,0

Integrante 1: 0,40 x 78,0 + 0,60 x 75,0 = 76,2

Integrante 2: 0,40 x 72,0 + 0,60 x 75,0 = 73,8

Os dois integrantes de uma mesma dupla podem terminar com notas diferentes, e a diferença vem inteiramente dos vinte pontos individuais de cada avaliação. Isso é deliberado: a dupla constrói junto, mas a arguição é de cada um.

9.5. O que zera um item, independentemente do resto

- Capacidade 8 no nível Insuficiente quando o sistema para de funcionar com o modelo removido. Degradação graciosa não é um detalhe do item: é o item.
- Apresentação no nível Insuficiente quando a dupla não apresenta na data do rodízio, ou quando os slides não foram depositados no prazo.
- Contribuição individual no nível Insuficiente quando não há commits com autoria institucional atribuída ao integrante, ou quando todos os seus commits estão concentrados na véspera da entrega.
- Qualquer capacidade no nível Insuficiente quando a evidência apresentada não é reproduzível: número sem comando, sem dado e sem quem gerou não conta.

10. O que entregar

10.1. Envio

Um e-mail por dupla/indivíduo, enviado do endereço institucional de um dos integrantes para arthur.miranda@ufla.br, **até as 23h59 da terça-feira (20 de outubro) ou (8 de dezembro)**. Mensagem enviada de endereço pessoal não é considerada recebida.

- Assunto: [GAT108] Fragmented Learning – AV1 ou AV2 – Dupla Nomes / Nome Indivíduo
- Corpo: nomes completos, o que cada integrante implementou nesta fase, link do repositório e hash do commit entregue
- Anexo: relatório em PDF, até 15 páginas sem contar apêndices

A entrega é da dupla, mas a nota tem componente individual. É responsabilidade de cada integrante confirmar que sua contribuição está corretamente atribuída no corpo do e-mail e no histórico do repositório.

10.2. Estrutura obrigatória do relatório

Todas as avaliações seguem esta estrutura. Seções ausentes são pontuadas como zero no critério correspondente.

§	Seção	Conteúdo mínimo
1	**Identificação**	No cado de dupla, integrantes, papel de cada um nesta fase, link do repositório, hash do commit entregue.
2	**O que foi implementado**	Uma subseção por integrante, listando classes e métodos que ele escreveu e os commits correspondentes. Sem isso não há nota individual.
3	**Fundamentação**	O conteúdo teórico da fase aplicado ao que vocês fizeram. Não é resumo de aula: é a justificativa das decisões tomadas.
4	**Evidências**	Figuras, tabelas e números exigidos pela fase (ver 10.3). Toda figura tem legenda e é citada no texto.
5	**Integração**	O que foi consumido de outras duplas, quais fakes ainda estão em uso e por quê, e quais discrepâncias apareceram na troca.
6	**Autoria por integrante**	Uma subseção por integrante, listando as classes e os métodos que ele escreveu e os commits correspondentes. Sem isso não há nota individual de contribuição.
7	**Limitações**	O que não funciona, o que funciona mal e o que vocês não conseguiram medir. Reconhecer limitação pontua; escondê-la penaliza.
8	**Declaração de uso de IA**	Quais ferramentas de IA foram utilizadas na construção, em que partes do trabalho, e o que cada integrante verificou por conta própria.
9	**Leitura em camadas**	Somente na AV2. Uma página sobre comoditização: quais camadas da pilha o sistema de vocês efetivamente tomou, em quais permaneceu dependente, e onde a implementação fica aquém do que o Anexo G descreve. Use a Figura 13 como referência e seja honesto nas duas direções.

10.3. Evidências exigidas em cada avaliação

A seção 4 do relatório deve conter, obrigatoriamente, o que a tabela abaixo lista. Evidência ausente derruba o item correspondente da subseção 9.2 ou 9.3 para o nível Insuficiente, ainda que o código funcione.

AV1	**Capturas das telas implementadas. Log de aquisição com pelo menos trinta minutos contínuos e aparelho bloqueado. Saída do comando de teste. Tabela com dez execuções do teste de concorrência, uma linha por execução. Gráfico de dispersão das leituras estáticas e a razão de otimismo do sensor. Tabela do resíduo antes e depois da reconciliação em pelo menos três janelas reais. Demonstração de que sem o detector de erro grosseiro a reconciliação piora o resultado. Tabela de sensibilidade da detecção à degradação injetada, com a semente declarada.**
AV2	Tabela de tarefas com período, Ci medido com dispersão, prazo, prioridade e tempo de resposta calculado, com a conclusão de escalonabilidade. Padrão recorrente descoberto e métrica de predição em dados retidos, com a data de congelamento da separação. Catálogo de ferramentas com esquema e taxa de saída inválida sobre pelo menos cem decisões. Tabela comparativa entre o motor determinístico e o motor com modelo nos mesmos dados. Log evidenciando o recuo seguro e a abstenção em ação. Amostra de um registro cifrado e do mesmo registro decifrado. Curva da grandeza física contra o parâmetro que ela reconfigura. Tabela da frota com deriva medida entre os nós e um evento fundido, antes e depois. Tabela TRL nível a nível. Gráfico de speedup com dispersão de cinco repetições e a fração paralelizável estimada. Tabela do ensaio cego com as quatro métricas para cada motor.

10.4. Demonstração

A demonstração e a arguição acontecem ao vivo, nas duas apresentações de projeto, com a turma presente. Não há nada a gravar, a editar ou a enviar.

Cada avaliação tem um momento presencial obrigatório de demonstração. A dupla/indivíduo roda o sistema diante da turma, sobre dados reais ou reproduzidos, e responde às perguntas.

Avaliação	Quando	Duração	O que precisa aparecer na tela
AV1	segunda e terça (19 e 20/10)	9 min, dentro da exposição	Aplicativo adquirindo com o aparelho bloqueado e contagem de fixos crescendo; reprodução acelerada com injeção de falha; resíduo antes e depois da reconciliação com números na tela; erro grosseiro sendo detectado.
AV2	segunda e terça (7 e 8/12)	9 min, dentro da exposição	Lugares descobertos e predição sobre um dia retido; agente decidindo com o JSON da chamada visível; DESLIGAR o modelo em execução e mostrar o sistema seguindo pelo RuleEngine; registro cifrado e decifrado; malha térmica atuando; frota com dois ou mais aparelhos e um evento fundido; resultados do ensaio cego e do speedup.

Em todas elas, os últimos minutos são de arguição: cada integrante responde a pelo menos uma pergunta sobre o que assinou. Integrante ausente não recebe a parcela individual daquela avaliação.

11. As apresentações

São três momentos de apresentação no semestre, e nos cinco encontros que eles ocupam não há conteúdo de aula: a proposta (31/08 e 1/09), a Apresentação I e a Apresentação Final. As duas últimas valem doze pontos cada, e são pontos individuais.

11.1. O rodízio

Momento	Quando	Quem apresenta	Formato
Proposta	31/08 e 01/09	10 m mesma sessão	Exposição e 5m de arguição
Apresentação I	Segunda	A, B e C	22 min de exposição e 8 de arguição
Apresentação I	Terça	D, E e F	22 min de exposição e 8 de arguição
Apresentação Final	Segunda	D, E e F	22 min de exposição e 8 de arguição
Apresentação Final	Terça	A, B e C	22 min de exposição e 8 de arguição

A ordem se inverte entre a Apresentação I e a Final.

11.2. O que uma apresentação precisa conter

Esta é a parte que mais gera dúvida, então está escrita como roteiro. Os vinte e dois minutos de exposição se dividem em três blocos, e nenhum deles é opcional.

Tempo	Bloco	O que precisa aparecer
8 min	**Fundamentação: como vocês chegaram ao resultado**	O raciocínio técnico, explicitamente ligado ao conteúdo da disciplina. Qual restrição física do domínio sustenta a reconciliação e por que ela é linear. Por que aquela estrutura de dados e não outra, com a complexidade das operações. Onde está a região crítica e por que ali. Que política de escalonamento foi assumida. Isto não é revisão de aula: é a aula aplicada ao problema de vocês, com as escolhas justificadas e as alternativas descartadas nomeadas.
9 min	**O MVP ao vivo**	O sistema rodando, no aparelho, diante da turma. Não é captura de tela nem gravação: gravação só é aceita como plano de contingência declarado, e limita

Tempo	Bloco	O que precisa aparecer
		o item ao nível Adequado. Mostre o caminho completo do dado, do sensor ao alerta, e provoque ao menos uma falha de propósito para mostrar o comportamento nos limites.
5 min	**Resultados quantitativos**	Os números da fase, com a condição de medição declarada: quanto o resíduo caiu, em quantas janelas; quantas execuções do teste de corrida e quantas passaram; qual o Ci medido e a dispersão; qual a acurácia da predição fora do treino. Número sem comando, sem dado e sem quem gerou não conta.
8 min	**Arguição**	Perguntas da turma e do docente. CADA integrante responde a pelo menos uma. Responder que não sabe e que vai verificar é aceitável; inventar não é.

A diferença entre a Apresentação I e a Final
Não é de formato, é de escopo. O MVP cobre a camada de aquisição e estimação: o sistema sente, corrige e detecta. Nas fases posteriores, ele cobre tudo: aprende, decide, abstém-se, cifra e coopera com outro nó. A fundamentação acompanha: na primeira, reconciliação e concorrência; na segunda, escalonabilidade, aprendizado, agente e o que o modelo agregou ou não.

11.3. Regras

- No máximo 11 slides. O primeiro com o título e nomes. Em seguida, já é conteúdo.
- Os dois integrantes falam, e cada um assume ao menos um dos três blocos de exposição.
- Os slides são depositados em docs/presentations/ no repositório da dupla/indivíduo até o domingo anterior. Sem depósito, o item cai para Insuficiente.
- O tempo é cronometrado. Aos vinte e dois minutos a exposição termina, no meio da frase se for o caso, e começam os oito minutos de arguição.
- Prepare o ambiente antes: aparelho carregado, registro carregado, aplicativo instalado, e o segundo aparelho pareado quando a frota fizer parte da demonstração.

11.4. Como a apresentação é avaliada

Doze pontos em cada avaliação, apurados em um dos quatro níveis da subseção 9.1: zero, seis, nove ou doze pontos. Os cinco critérios abaixo determinam o nível. Dois critérios no nível Insuficiente impedem o nível Bom.

Critério	Insuficiente	Adequado	Bom	Excelente
Fundamentação ligada à disciplina	Descreve o que fez, sem justificar.	Cita o conceito da aula sem aplicá-lo ao domínio.	Justifica as escolhas com o conceito aplicado ao domínio.	Nomeia a alternativa descartada e diz por quê, com evidência.
MVP ao vivo	Não roda ou não foi preparado.	Roda com percalços, ou por gravação de contingência.	Roda ao vivo e percorre o caminho completo do dado.	Roda ao vivo e provoca uma falha de propósito, mostrando o comportamento no limite.
Resultados quantitativos	Impressões, sem dados.	Dados soltos, sem condição de medição.	Dados com condição declarada e reproduzíveis.	Interpretação dos dados e limitação reconhecida.
Arguição	Algum integrante não	Responde	Os dois respondem	Distinguem o que sabem do que precisam verificar, e

Critério	Insuficiente	Adequado	Bom	Excelente
individual	responde.	parcialmente.	com segurança.	trazem depois.
Uso do tempo e clareza	Estoura, ou sobra muito tempo.	Cabe no tempo, sem foco.	Cabe no tempo, com os três blocos equilibrados.	Escolhe deliberadamente o que deixar de fora e diz por quê.

É pontuação individual

Cada integrante recebe pelo bloco que apresentou e pela resposta que deu na arguição, e os dois podem receber notas diferentes pela mesma apresentação. Integrante que não fala e não responde recebe zero no item, ainda que a dupla tenha apresentado bem.

12. Problemas comuns

O Gradle sync falha na primeira abertura.

Quase sempre é rede ou proxy. Em File > Settings > Build > Gradle, confirme que o JDK apontado é o 17 empacotado com a IDE.

Os testes não aparecem no Android Studio.

Verifique se estão em app/src/test/java (unidade, roda na JVM) e não em app/src/androidTest/java (instrumentado, precisa de aparelho). Todos os testes deste projeto são de unidade.

A aquisição para quando bloqueio a tela.

Confirme três coisas: permissão de segundo plano concedida; serviço declarado com foregroundServiceType location; e o fabricante não aplicando otimização agressiva de bateria. Xiaomi, Samsung e Motorola exigem liberação manual nas configurações do aplicativo.

O teste de concorrência passa às vezes e falha às vezes.

Isso é um defeito, não instabilidade do teste. Você tem uma condição de corrida. Revise se todos os caminhos de leitura e escrita estão protegidos e se as cópias são feitas dentro da região crítica.

O resíduo não cai depois de reconciliar.

Na quase totalidade dos casos é sinal trocado na matriz de restrições, ou dt em milissegundos onde deveria ser segundos.

Minha PR (Pull Rrequest) está com conflito.

Faça rebase sobre a linha principal e resolva localmente. Numa dupla, conflito quase sempre significa que os dois mexeram no mesmo arquivo sem combinar; combinem antes de abrir o ramo.

A CI (Integração Contínua) está vermelha e não fui eu.

Descubra qual commit quebrou e corrija antes de seguir. Numa dupla de dois, a resposta é sempre um de vocês.

O push é recusado por tamanho de arquivo.

Alguém commitou o modelo. O .gitignore já o exclui, mas se entrou antes o histórico está contaminado. Avise o docente imediatamente e não tente reescrever o histórico por conta própria.

O modelo não carrega no emulador.

Provavelmente a biblioteca nativa não tem binário para a arquitetura do emulador. Confira com adb shell getprop ro.product.cpu.abilist. Use o RuleEngine, registre no relatório e siga.

O aplicativo trava quando o modelo responde.

A inferência está rodando na thread da interface. Use o InferenceExecutor. O Android encerra aplicações que bloqueiam a thread principal.

O modelo devolve texto em vez de JSON.

Esperado. Extraia o trecho entre a primeira chave de abertura e a última de fechamento, valide e conte a falha. É para isso que o OutputValidator existe.

Os aparelhos não se enxergam na frota.

Não use a rede da universidade: ela isola clientes. Um aparelho abre ponto de acesso e os demais conectam nele.

13. Glossário

Fix. Uma medição de posicionamento com instante, coordenadas, velocidade e as acurácias informadas pelo sensor.

Região crítica. Trecho de código que acessa recurso compartilhado e cuja execução por mais de uma thread ao mesmo tempo produz estado inconsistente.

Reconciliação de dados. Ajuste de medições redundantes para que satisfaçam restrições conhecidas, ponderando cada medida pela sua incerteza.

Resíduo. Quanto as medições violam a restrição. Reconciliar corretamente leva o resíduo a zero.

Aprendizado online. Aprendizado que atualiza o modelo a cada observação, com memória constante, sem reter o histórico.

Agente. Laço de software que percebe o estado, decide uma ação, executa e observa o resultado. Não é o modelo de linguagem: é o código em torno dele.

Ferramenta. Ação que o agente pode executar, com esquema declarado. É o único caminho pelo qual o modelo influencia o sistema.

Prefill. Fase de processamento do prompt antes do primeiro token gerado. Cresce com o tamanho do prompt e é a principal fração serial da inferência.

TTFT. Tempo até o primeiro token. Métrica de latência percebida da inferência.

Degradação graciosa. Propriedade de continuar operando com capacidade reduzida quando um componente falha, em vez de parar.

Rate Monotonic. Política de escalonamento por prioridade fixa em que tarefas de período menor recebem prioridade maior.

Deriva de relógio. Diferença entre os relógios de dois nós. Sem corrigir, o mesmo evento observado por dois aparelhos parece dois eventos.

TRL. Escala de 1 a 9 que classifica a maturidade de uma tecnologia, da concepção à operação comprovada.

Fake. Implementação falsa de um componente, fornecida pelo docente, para que nenhuma dupla fique bloqueada.

14. Referências

14.1. Enquadramento do projeto

de MIRANDA NETO, Arthur. The Cost Gradient of the Build: How Differential Commoditization Reshapes Entrepreneurship and Valuation — A Layer-Decomposed Risk Premium for the Post-AI Firm. Universidade Federal de Lavras, 2026. ISBN 978-65-02-13475-7. SSRN 6818518. Disponível em: https://ssrn.com/abstract=6818518.

de MIRANDA NETO, Arthur. Central Brains and the Decentralization Frontier: Institutional Sovereignty Through a Decentralized Protocol — A Seven-Layer Commoditization Analysis of Large Language Model Systems. Universidade Federal de Lavras, 2026b. Working draft. Citado ao longo deste documento como Miranda Neto (2026b); https://commoditizationstack.org/resources; mapa de leitura no Anexo G.

de MIRANDA NETO, Arthur. Data-Driven Platform of Platforms to Drive Business and ESG Practices in Decentralized Supply Chains. SSRN 6735341, 2025. Disponível em: https://ssrn.com/abstract=6735341.

14.2. Contraleitura

KOKOTAJLO, D.; ALEXANDER, S.; LARSEN, T.; LIFLAND, E.; DEAN, R. AI 2027. AI Futures Project, 2025. Disponível em: https://ai-2027.com. Cenário, não resultado revisado por pares.

EPOCH AI. How Far Can Decentralized Training Over the Internet Scale? Gradient Updates, 2025.

14.3. Âncoras técnicas do projeto

Estas são as fontes primárias por trás dos componentes que vocês implementam. Use-as no aprofundamento da fundamentação, e não apenas o slide da aula.

BROWN, T. B. et al. Language Models are Few-Shot Learners. NeurIPS 33, 2020. arXiv:2005.14165. Origem do aprendizado em contexto: o modelo condiciona, não aprende.

LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 33, 2020. arXiv:2005.11401.

BANG, F. GPTCache: An Open-Source Semantic Cache for Large Language Model Applications. NLP-OSS Workshop, ACL, 2023. Base da extensão de cache semântico da AV2.

ANGELOPOULOS, A. N.; BATES, S. A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification. arXiv:2107.07511, 2021. Abstenção calibrada, contra a qual a abstenção por limiar deve ser comparada.

CASTRO, M.; LISKOV, B. Practical Byzantine Fault Tolerance. OSDI, 1999. Referência do consenso 3f+1 que a fusão de observações do projeto NÃO implementa.

McMAHAN, H. B. et al. Communication-Efficient Learning of Deep Networks from Decentralized Data. AISTATS, 2017. arXiv:1602.05629. Aprendizado federado, fora do escopo desta disciplina.

NARASIMHAN, S.; JORDACHE, C. Data Reconciliation and Gross Error Detection: An Intelligent Use of Process Data. Gulf Publishing, 2000. Base da Fase I.

BURNS, A.; WELLINGS, A. Real-Time Systems and Programming Languages. 4. ed. Addison-Wesley, 2009. Base da análise de escalonabilidade.

Anexo A — Catálogo das dez capacidades

Este é o contrato da disciplina. São as dez capacidades, no domínio que propôs. A coluna de competência e a coluna de conexão vêm da Tabela C1 de Miranda Neto (2026b), Apêndice C; a coluna de critério de aceitação é o que será verificado nas avaliações.

#	Capacidade	Competência formada	Critério de aceitação	No paper
1	**Domínio, contratos e estruturas**	projeto por contrato; estruturas de capacidade fixa e sua complexidade	O modelo de domínio valida invariantes e lança exceção específica. A estrutura de trajeto responde comprimento, média e recorte temporal em tempo linear e memória constante.	Seções 3 e 4
2	**Incerteza e reconciliação**	reconciliação: covariância a partir das acurácias, mínimos quadrados ponderados, detecção de erro grosseiro	A covariância é montada com a acurácia que o sensor informa, não com valor arbitrado. O resíduo cai em três janelas de dados reais. Um erro grosseiro injetado é detectado, e a dupla demonstra a reconciliação piorando o resultado quando o detector é removido.	Seções 7 e C.3
3	**Concorrência e região crítica**	exclusão mútua, região crítica e evidência experimental de ausência de corrida	A estrutura compartilhada devolve cópias, nunca a referência interna. Dez execuções do teste de corrida, uma linha por execução, sem falha.	Apêndice A
4	**Aquisição periódica em tempo real**	tarefas de período fixo com serviço de primeiro plano e permissões corretas	A aquisição continua com o aparelho bloqueado por trinta minutos contínuos, demonstrado ao vivo, com a grandeza física do domínio proposto.	Seção 6.2, Apêndice A
5	**Ensaio reprodutível e injeção de falhas**	método experimental: semente fixa, reprodutibilidade e degradação controlada	Registro gravado, reproduzido de forma determinística e degradado com ruído, perda e salto. Tabela de sensibilidade mostrando onde a detecção começa a falhar.	Seção 14
6	**Instrumentação e escalonabilidade**	sistemas de tempo real: tempo de execução medido, escalonabilidade por prioridade fixa	Tabela de tarefas com Ci medido e dispersão declarada, tempo de resposta calculado e conclusão explícita sobre escalonabilidade. Estimativa no olho não conta.	Seções C.5 e 6.2
7	**Aprendizado online sob memória constante**	estimação em fluxo: o modelo atualiza a cada observação, sem reter histórico	O sistema descobre um padrão recorrente do domínio e prevê algo verificável. A predição é validada em dados retidos, com métrica declarada, e não nos dados de treino.	Seções 3.6 e 13
8	**Agente e modelo**	agente: perceber,	O laço respeita orçamento de	Seções 8

#	Capacidade	Competência formada	Critério de aceitação	No paper
	substituível	decidir, validar, agir e fechar a malha; degradação graciosa como teste de aprovação	latência, valida saída estruturada, abstém-se quando a corroboração é fraca e nunca propaga exceção. Com o modelo removido em execução, o sistema continua pelo motor determinístico. Este é o critério que mais reprova.	e C.2
9	**Criptografia e frota multi-nó**	criptografia autenticada e manejo de chave; deriva de relógio e confirmação multi-nó sem nuvem	Dado sensível cifrado com IV variável e chave no armazenamento seguro do aparelho, com modelo de ameaça escrito. Dois ou mais nós da própria dupla (ou com colegas) cooperando sobre ponto de acesso, deriva de relógio medida, e um evento observado por mais de um nó reconciliado em alerta único. Três nós é o nível Excelente.	Seções C.4 e A.7
10	**Validação cega, TRL e Lei de Amdahl**	medição: se o modelo agrega valor contra a linha de base determinística	Ensaio cego sobre dados retidos comparando o agente com modelo e o agente só com regras, nas mesmas quatro métricas. Speedup com cinco repetições e dispersão. Classificação TRL justificada nível a nível.	Seções C.6 e 14
Como ler a coluna de critério: Ela está escrita em termos do que se demonstra, não do que se implementa. Ao preencher a ficha de proposta, traduza cada critério para o seu domínio e diga qual teste automatizado o verifica.				

Anexo B — Modelo da ficha de proposta

Comite como docs/PROPOSAL.md no repositório da dupla, até o domingo anterior ao quinto encontro. Três a quatro páginas bastam. A tabela da parte 3 é o que decide o parecer.

Proposta de trabalho — Dupla X
- Integrantes: - Data:
- Domínio escolhido:
1. O problema
Quem tem o problema. Por que ele dói. O que se faz hoje na falta de solução.
Cinco linhas. Se precisar de mais, o problema ainda não está claro.
2. O que é observado
- Ativo ou processo:
- Grandeza física medida:
- Sensor e origem da incerteza (o que o sensor informa sobre a própria acurácia):
- Período de amostragem pretendido e prazo associado:
3. Mapeamento das dez capacidades
#	Capacidade	O que significa no nosso domínio	Teste de aceitação
1	Domínio, contratos e estruturas		
2	Incerteza e reconciliação		
3	Concorrência e região crítica		
4	Aquisição periódica em tempo real		
5	Ensaio reprodutível e injeção de falhas		
6	Instrumentação e escalonabilidade		
7	Aprendizado online sob memória constante		
8	Agente e modelo substituível		
9	Criptografia e frota multi-nó		
10	Validação cega, TRL e Lei de Amdahl		
4. Dados
- Como serão obtidos:
- Quantos dias ou quantas execuções:
- Como a separação treino e validação será congelada, e em que data:
5. Riscos
| Risco | Probabilidade | Plano B |

Parecer do docente
- [ ] Aprovada - [ ] Aprovada com ressalva - [ ] A refazer
- Observações:

O erro mais comum ao preencher a parte 3 - Mapeamento das dez capacidades

Escrever o que se pretende implementar em vez do que se pretende demonstrar. "Vamos usar reconciliação de dados" não é mapeamento; "a velocidade integrada deve igualar o deslocamento medido, e o teste verifica que o resíduo cai em três janelas reais" é mapeamento. A diferença entre as duas frases é a diferença entre uma proposta aprovada e uma a refazer.

Anexo C — Domínios inspiracionais

Dez problemas que comportam as dez capacidades. São ponto de partida, não cardápio fechado: uma dupla/indivíduo pode propor o décimo primeiro, e vários destes são melhores quando recortados para um caso concreto que você conheça. Vários foram escolhidos por proximidade com a região e com o curso.

Domínio	Grandeza medida	Onde estão o aprendizado e a reconciliação
Vibração de máquina rotativa	acelerômetro de três eixos	Assinatura espectral normal de um motor ou bomba; desvio indica desbalanceamento ou folga. Reconciliação entre eixos e entre nós posicionados em pontos distintos da mesma máquina.
Cadeia de frio em transporte	temperatura e posição	Envelope térmico normal do trajeto; excursão fora da faixa é o evento. A restrição física liga taxa de variação térmica a abertura de porta e a tempo parado.
Irrigação e microclima em cultura	umidade, temperatura, posição	Padrão diário e sazonal do talhão. Vários nós no mesmo talhão reconciliam leituras de sensores baratos e discordantes.
Transporte coletivo urbano	posição e velocidade	Tempo de percurso normal por linha, por hora e por dia da semana; atraso além do percentil histórico é o evento. Próximo do caso de referência.
Consumo elétrico de instalação	corrente ou potência	Perfil de carga normal por hora e por dia; anomalia indica equipamento ligado fora de hora ou defeito. A restrição liga energia acumulada à integral da potência.
Ruído urbano	nível de pressão sonora	Mapa temporal do ruído de um trecho; a frota de nós cobre uma área e reconcilia medições de microfones não calibrados.
Segurança de trabalhador isolado	acelerômetro, posição, tempo parado	Padrão normal de deslocamento e de postura; queda ou imobilidade prolongada é o evento. Exige cuidado redobrado com o modelo de ameaça e com a LGPD.
Qualidade de pavimento	acelerômetro e posição	Assinatura de vibração normal do trecho; buraco ou irregularidade é o evento. Vários veículos passando pelo mesmo trecho reconciliam a mesma anomalia.
Ocupação e conforto de ambiente	temperatura, umidade, luz, ruído	Padrão de ocupação normal de uma sala; o agente reconfigura a taxa de amostragem conforme a ocupação, fechando a malha.
Deslocamento pessoal	posição e velocidade	É o caso inspirador sugeridos, o Fragmented Learning. Escolhê-lo é legítimo e reduz o risco, mas a dupla assume o domínio, não copia o código.

No escopo de Engenharia de Controle e Automação, e adequados a alunos do sétimo período, também são boas escolhas: bancada de motor com inversor (vibração e temperatura como envelope normal, com desbalanceamento como evento); malha de controle PID executada no aparelho como controlador de borda, com o agente reconfigurando ganhos sob restrição; monitoramento de vibração de máquina rotativa; qualidade de energia (tensão, corrente e harmônicos) em uma instalação; dosagem ou limpeza em processo (CIP), com vazão e nível reconciliados; e veículo guiado automaticamente (AGV) ou robô móvel, com odometria e detecção de obstáculo. Todos comportam as dez capacidades e a camada física exigida.

Domínios que costumam ser recusados

Aplicações puramente de interface, sem grandeza física medida: não há camada 1 nem reconciliação. Problemas de classificação de imagem: o esforço migra para um modelo que vocês não vão treinar, e as capacidades 2, 6 e 9 ficam sem lugar. Sistemas de recomendação sem sensor: não há tempo real. E qualquer domínio em que a coleta de dados dependa de autorização de terceiros que não esteja garantida.

Anexo D — Especificação de contratos e testes exigidos

O repositório é https://github.com/admufla/AA2026_2. Cada dupla/indivíduo constrói o próprio sistema a partir desta especificação: os contratos que precisa implementar e os testes que precisa escrever, no seu domínio. Os nomes abaixo são uma decomposição sugerida; você pode renomear, dividir, fundir ou substituir qualquer contrato, desde que registre em docs/DECISIONS.md o que mudou e por quê e mantenha o teste equivalente. O que se avalia é a capacidade demonstrada, não o nome do arquivo. O caso inspirador da seção 2.3 ilustra como esses contratos se encaixam.

D.1. Contratos a implementar

Contrato	Responsabilidade que você implementa
Domínio (Fix, Track, Place, Event)	Tipos imutáveis do seu domínio, com invariantes validadas na construção e uma exceção específica para violação.
FixSource	Leitura do sensor do seu domínio, preservando a acurácia que ele informa. No caso inspirador é o GPS; no seu, o sensor equivalente.
Leitor de registros	Interpretação do formato em que você grava os dados, para o ensaio reprodutível.
Reconciler	Reconciliação sob restrição física: mínimos quadrados ponderados, com covariância vinda da acurácia do sensor, e cálculo de resíduo.
GrossErrorDetector	Detecção de erro grosseiro por resíduo normalizado, com critério quantitativo de corte.
Buffer e serviço de aquisição	Buffer circular protegido (região crítica) e serviço de aquisição periódica em tempo real, com período fixo, resistente ao aparelho bloqueado.
FaultInjector	Injeção de falhas determinística, com semente fixa, para o ensaio reprodutível.
Profiler e análise de escalonabilidade	Instrumentação com média, percentis e máximo, e análise de período, prazo, jitter e utilização.
Aprendizado e predição	Agrupamento incremental e predição do próximo estado, em memória constante, com métrica validada fora da amostra.
Agente	Laço que decide, valida a saída, age e abstém, com um motor de linguagem substituível atrás de uma interface e degradação graciosa.
Segurança e frota	Cifra autenticada em repouso, chave em armazenamento seguro, e transporte mínimo entre nós para reconciliação e deriva de relógio.
Validação cega	Separação entre ajuste e avaliação, métricas e experimento de Amdahl.

Componentes de infraestrutura: resolva cedo, mas resolva você

O serviço em primeiro plano com canal de notificação, o fluxo de permissões na ordem correta, a álgebra de matrizes e o transporte entre aparelhos consomem tempo sem pontuar por si. Não há versão pronta: implemente-os cedo, com o mínimo necessário, e não gaste o semestre reescrevendo encanamento. Andaimes e implementações falsas que você criar para destravar uma camada não podem estar em uso na entrega final sem declaração no relatório.

D.2. Testes exigidos, por capacidade

Você escreve os testes; a especificação fixa o rigor. Cada bateria abaixo é o mínimo para o nível Bom do item correspondente do Anexo A.

Foco	Rigor exigido do teste que você escreve
Domínio e contratos	Invariantes rejeitadas na construção; exceção específica; estrutura em memória constante.
Reconciliação	Resíduo reduzido em três janelas reais; erro grosseiro detectado; verificação quantitativa, não visual.
Concorrência	Teste de corrida executado dez vezes sem falha, registrado uma a uma; cópias defensivas.
Aquisição periódica	Trinta minutos contínuos com o aparelho bloqueado, demonstrados ao vivo.
Ensaio reprodutível	Semente fixa, reprodução determinística e tabela de sensibilidade da detecção.
Escalonabilidade	Tabela de tarefas com números medidos; nenhum prazo perdido sob carga.
Aprendizado	Acurácia preditiva medida fora da amostra de treino; memória constante ao longo do tempo.
Agente	O sistema continua funcionando com o modelo removido ou trocado.
Segurança e frota	Nenhum dado pessoal em claro; reconciliação entre nós com deriva de relógio corrigida.
Validação cega	Ajuste e avaliação separados; métricas e ganho de Amdahl reportados.

Adaptar um conjunto de testes ao seu domínio ou mantê-lo sobre o caso inspirador e escrever os seus em paralelo é decisão da dupla; registre-a em docs/DECISIONS.md.

D.3. A sequência sugerida de construção, semana a semana

As figuras abaixo mostram uma sequência sugerida de construção, semana a semana. Verde escuro é o que construir naquela semana, verde claro é o que já foi construído, cinza é o que ainda não. Use-as para localizar onde você está; no seu domínio, os nomes mudam e as posições não.

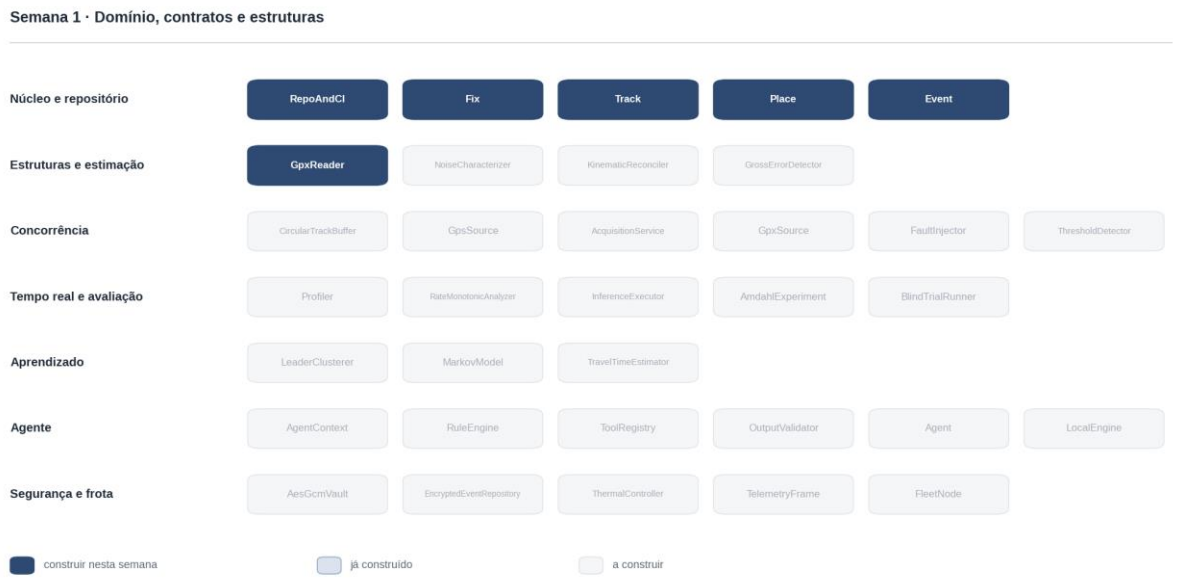


Figura 3 — a sequência sugerida de construção na semana 1

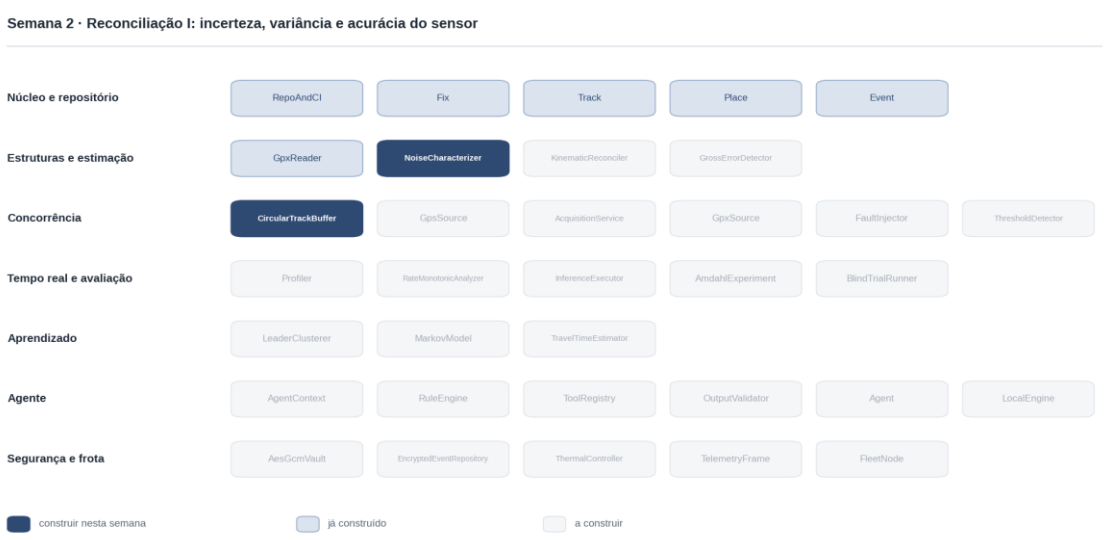


Figura 4 — a sequência sugerida de construção na semana 2



Figura 5 — a sequência sugerida de construção na semana 3



Figura 6 — a sequência sugerida de construção na semana 4

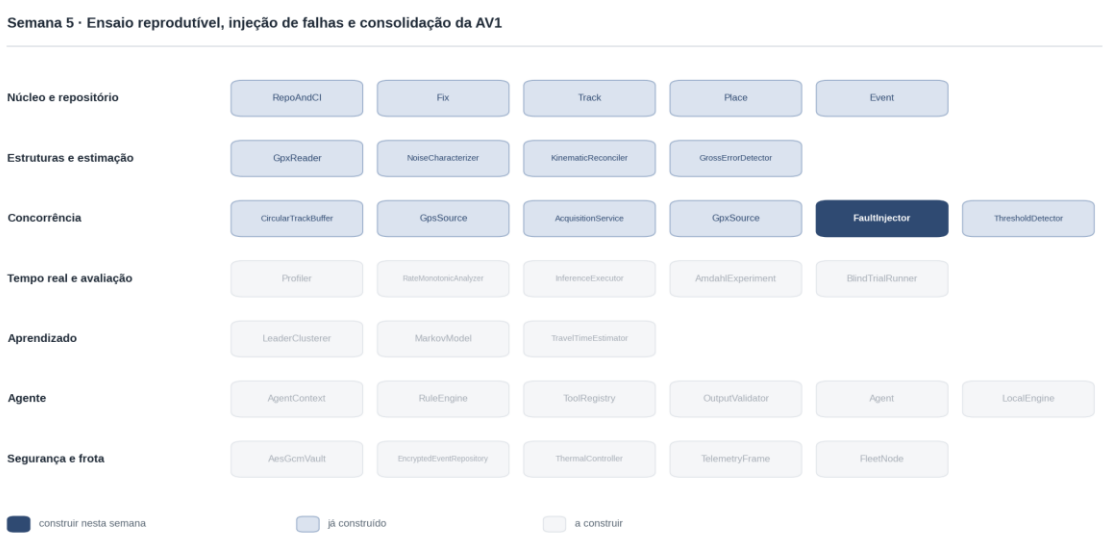


Figura 7 — a sequência sugerida de construção na semana 5



Figura 8 — a sequência sugerida de construção na semana 6



Figura 9 — a sequência sugerida de construção na semana 7

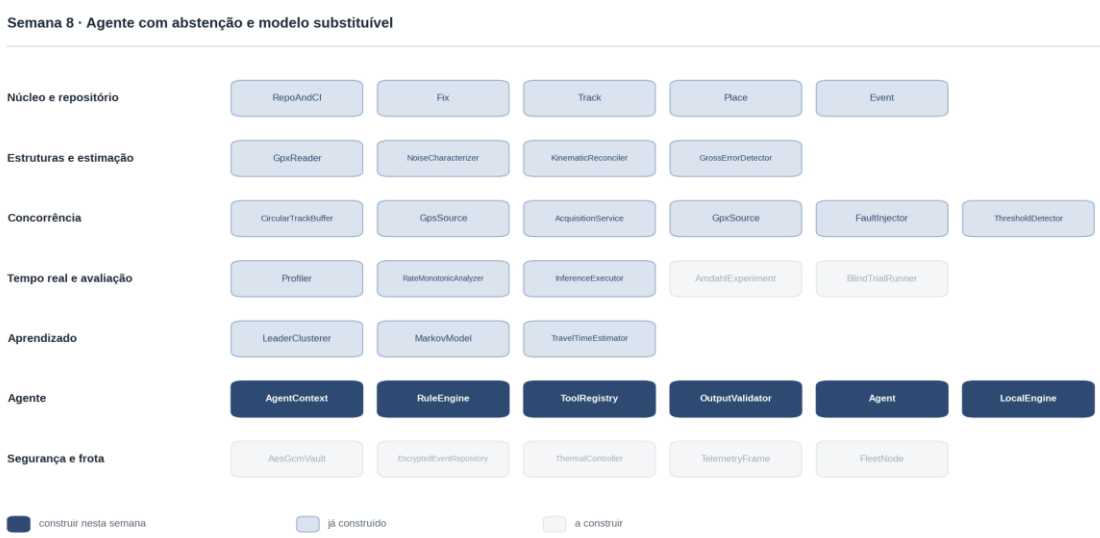


Figura 10 — a sequência sugerida de construção na semana 8

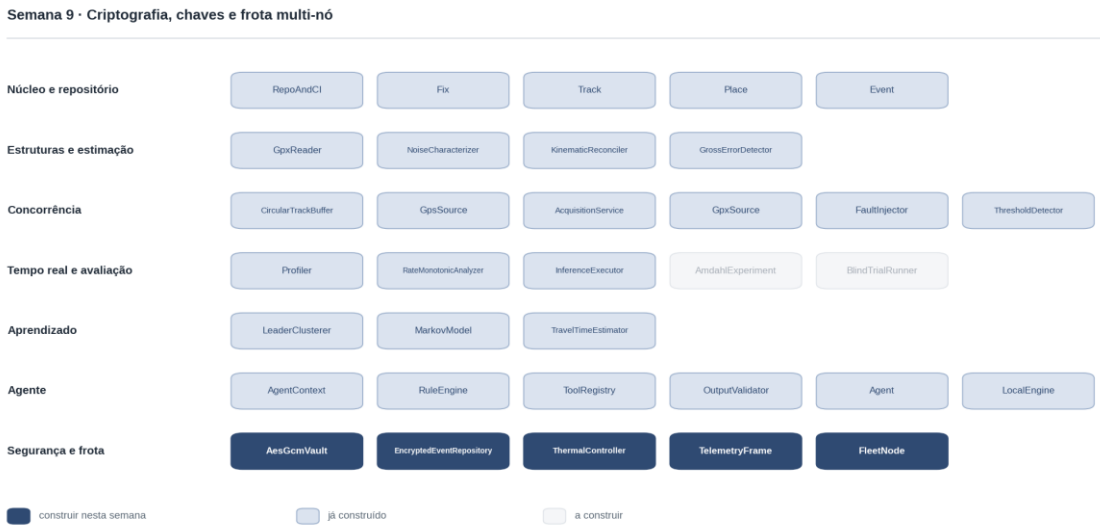


Figura 11 — a sequência sugerida de construção na semana 9



Figura 12 — a sequência sugerida de construção na semana 10

Anexo E — Lista de verificação antes de cada entrega

Percorra esta lista na sexta-feira anterior à entrega. Cada linha corresponde a um item pontuado nas subseções 9.2 e 9.3; um "não" é um item que vai cair de nível.

E.1. Antes da AV1

	Verificação	Item que ela alimenta
☐	A ficha de proposta está aprovada e o que ela declarou é o que estamos entregando	Proposta, 12 pt
☐	O modelo de domínio valida invariantes e lança exceção própria	Capacidade 1, 8 pt
☐	As estruturas respondem em memória constante e há teste que prova	Capacidade 1, 8 pt
☐	A covariância vem da acurácia que o sensor informa, e não de valor arbitrado	Capacidade 2, 20 pt
☐	O resíduo cai em três janelas de dados reais, com a tabela no relatório	Capacidade 2, 20 pt
☐	Um erro grosseiro injetado é detectado, e mostramos a reconciliação piorando sem o detector	Capacidade 2, 20 pt
☐	O buffer devolve cópias e nunca a referência interna	Capacidade 3, 16 pt
☐	Rodamos o teste de corrida dez vezes e registramos cada execução	Capacidade 3, 16 pt
☐	A aquisição sobreviveu a trinta minutos com o aparelho bloqueado, com log	Capacidade 4, 8 pt
☐	A reprodução é determinística e a semente está declarada	Capacidade 5, 8 pt
☐	Há tabela de sensibilidade mostrando onde a detecção começa a falhar	Capacidade 5, 8 pt
☐	O relatório tem as nove seções da subseção 10.2 e cabe em quinze páginas	Relatório, 8 pt
☐	Ensaiamos os três blocos da apresentação e o MVP roda no aparelho carregado	Apresentação I, 12 pt
☐	Cada integrante tem commits com autoria @ufla.br atribuída	Individual, 8 pt
☐	Os commits de cada integrante estão distribuídos pelas semanas, e não concentrados na véspera	Individual, 8 pt
☐	A integração contínua está verde na linha principal	Relatório, 8 pt

E.2. Antes da AV2

	Verificação	Item que ela alimenta
☐	O Ci de cada tarefa é medido, com dispersão declarada, e não estimado	Capacidade 6, 12 pt
☐	O tempo de resposta foi calculado e há conclusão explícita de escalonabilidade	Capacidade 6, 12 pt
☐	O padrão recorrente aprendido faz sentido no domínio e está documentado	Capacidade 7, 16 pt
☐	A predição foi validada em dados retidos, e a data de congelamento está registrada	Capacidade 7, 16 pt
☐	O agente respeita o orçamento de latência e nunca propaga exceção	Capacidade 8, 20 pt
☐	A abstenção é acionada quando a corroboração é fraca, e demonstramos isso	Capacidade 8, 20 pt
☐	Removemos o modelo em execução e o sistema continuou pelo motor determinístico	Capacidade 8, 20 pt
☐	O IV varia a cada cifragem e a chave está no armazenamento seguro do aparelho	Capacidade 9, 12 pt
☐	O modelo de ameaça está escrito, e diz o que fica em claro e por quê	Capacidade 9, 12 pt

	Verificação	Item que ela alimenta
☐	Dois ou mais nós cooperaram, a deriva foi medida e um evento foi fundido em alerta único	Capacidade 9, 12 pt
☐	O ensaio cego tem as quatro métricas, para os dois motores	Capacidade 10, 12 pt
☐	O speedup tem cinco repetições por configuração e a dispersão está no gráfico	Capacidade 10, 12 pt
☐	A classificação TRL é justificada nível a nível	Capacidade 10, 12 pt
☐	O relatório final tem a seção de leitura em camadas, com a Figura 14	Relatório, 8 pt
☐	Respondemos por escrito se o modelo agregou alguma coisa, com números	Relatório, 8 pt
☐	Ensaiamos os três blocos da apresentação final e o segundo aparelho está pareado	Apres. Final, 12 pt
☐	Cada integrante tem commits próprios nesta fase, com autoria @ufla.br	Individual, 8 pt

A pergunta de controle

Se o docente abrisse o seu repositório hoje e rodasse os testes, ele conseguiria reproduzir cada número que está no seu relatório? Se a resposta for não para algum número, esse número não conta, e o item que ele sustenta cai para Insuficiente. Reprodutibilidade não é rigor acadêmico decorativo: é o que separa medição de impressão.

Anexo F — Roteiro de fundamentação das apresentações

Os oito minutos de fundamentação são a parte que mais falha, porque é tentador descrever o que se fez em vez de justificar por que se fez. Este anexo lista, para cada apresentação, as perguntas que a fundamentação precisa responder. Não é preciso responder a todas em oito minutos: escolha as três ou quatro em que o seu projeto tem mais a dizer, e diga bem.

F.1. Apresentação I

Conteúdo da disciplina	Perguntas que a fundamentação deve responder
Estrutura de dados	Por que uma estrutura de capacidade fixa e não uma lista que cresce? Qual a complexidade das três operações que vocês implementaram, e qual delas poderia ser melhor? Por que devolver cópia e não referência, e o que aconteceria sem isso?
Incerteza de medição	De onde vem a incerteza do seu sensor, e o que ele informa sobre a própria acurácia? A dispersão que vocês mediram bate com a acurácia declarada? Se não bate, o que isso significa para a matriz de covariância?
Reconciliação de dados	Qual é a restrição física do seu domínio, e por que ela é linear nas medidas? Por que a medida mais incerta recebe a maior correção? Em que caso a reconciliação piorou o resultado, e o que o detector de erro grosseiro fez a respeito?
Concorrência	Onde está a região crítica do seu sistema e por que ali? Que primitiva vocês usaram e qual descartaram? O que o teste de dez execuções prova, e o que ele não consegue provar?
Ensaio reprodutível	Por que a semente é fixa? Que modelos de degradação vocês injetaram e por que esses? Em que ponto a detecção começou a falhar, e isso é aceitável no seu domínio?

F.2. Apresentação Final

Conteúdo da disciplina	Perguntas que a fundamentação deve responder
Sistemas de tempo real	Quais são as tarefas periódicas do seu sistema, com que período e que prazo? O conjunto é escalonável, e como vocês souberam? Qual tarefa está mais perto do prazo, e de onde vem o jitter dela?
Aprendizado embarcado	Por que aprendizado online e não um ajuste sobre todo o histórico? O que se perde com memória constante? Como vocês congelaram a separação entre treino e validação, e por que antes de olhar o resultado?
Modelos na borda	Por que o modelo não pode entrar na malha crítica? O que é prefill e quanto ele custou no seu aparelho? Por que um modelo compacto erra o formato, e o que o validador faz com isso?
Agente e abstenção	Por que o agente é código e o modelo é peça? O que acontece quando vocês removem o modelo em execução? Quando o seu sistema se abstém, e por que abster-se é melhor do que alertar com evidência fraca?
Segurança	Qual é o modelo de ameaça do seu domínio? O que fica em claro no armazenamento e por quê? Por que o vetor de inicialização precisa variar a cada cifragem?
Sistemas ciberfísicos e frota	Que grandeza física realimenta o comportamento do software, e com que lei de controle? Por que os relógios dos nós divergem, e como vocês estimaram a deriva? O que a fusão de observações ganhou sobre a decisão isolada?
Desempenho e maturidade	Qual a fração paralelizável estimada, e por que ela difere entre configurações? Em que nível TRL o sistema está, e o que faltaria para o seguinte? E a pergunta que fecha tudo: o modelo agregou alguma coisa?

A pergunta que distingue o nível Excelente

Não é qual conceito vocês aplicaram, é qual alternativa vocês descartaram e por quê. Quem diz que usou mínimos quadrados ponderados está no nível Bom; quem diz que considerou ponderar de outro modo e por que aquilo teria distorcido a medida mais confiável está no Excelente. A escolha justificada, com a alternativa nomeada, é o que se procura.

Anexo G — O paper de referência e como usá-lo

Este projeto tem um enquadramento teórico explícito, e ele está publicado. O documento de referência é de Miranda Neto (2026b), **Central Brains and the Decentralization Frontier**, que analisa a soberania institucional em inteligência artificial pela decomposição em sete camadas de Miranda Neto (2026). Este anexo diz o que ler, o que não ler, como citar e como discordar.

Declaração de autoria

O paper é de autoria do docente da disciplina. Isso não o torna doutrina. Duas consequências práticas seguem, e valem como regra: nenhuma nota depende de concordar com ele, e toda apresentação que o utilize deve apontar ao menos um ponto em que a dupla discorda, em que o argumento está pouco sustentado, ou em que o caso do Fragmented Learning não se encaixa. Essa exigência está na rubrica da apresentação, no critério de fundamentação. Uma fundamentação que apenas devolve o texto do professor recebe o nível Adequado, não mais.

G.1. Por que este paper e não outro

Três decisões do projeto parecem, sem enquadramento, capricho do docente: executar tudo no aparelho, exigir que o sistema funcione sem o modelo de linguagem, e montar uma frota **(frota aqui representa multi-nó, assim a palavra frota é uma habilidade requerida, que você pode adequar a terminologia para seu problema)** sobre ponto de acesso em vez de usar um servidor. O paper mostra que as três decorrem de uma mesma posição, e nomeia o que se ganha e o que se perde com ela.

A tese, em uma frase: uma instituição não conquista soberania disputando a fronteira, que as paredes de memória e de comunicação vedam, mas tomando as camadas que comoditizaram. A soberania de operação e a de adaptação são alcançáveis hoje; a soberania de gênese não é. O paper é explícito quanto a essa instituição ser local por natureza, e nomeia entre elas a universidade, o município, o hospital e a cooperativa. Uma turma de dez semanas é a menor instância possível dessa figura.

G.2. Mapa de leitura

O paper tem setenta e três páginas, está em inglês e é dirigido a um público de pesquisa. Está disponível em https://commoditizationstack.org/resources (**aqui você também tem acesso ao texto em português**). Ler tudo, frio, produz paráfrase, que é o que a rubrica da apresentação pune. Leia por camadas.

Quem lê	O quê	Por quê
Todos, semana 1	Seções 3.5, 4.1 a 4.3 e 6.2 — cerca de seis páginas	É o núcleo conceitual do projeto: como o artefato congela, por que o modelo não aprende com a conversa, e qual é a parede de memória que limita o que cabe no aparelho. Sem isso, a reta final vira mágica.
Dupla que apresenta	A seção indicada no roteiro do Anexo F	Os oito minutos de fundamentação saem daí, e não do slide da aula. O Anexo F indica, por conteúdo da disciplina, as perguntas que a fundamentação deve responder.
Quem quiser, na AV2	Seções 12.2, 12.3, 13.5 e 15	Cache semântico, sombras replicadas, autoaperfeiçoamento e a análise de cenário. São a origem das extensões de nível Excelente e da discussão final do relatório.
Ninguém, como requisito	Apêndices A e B	São arquitetura de pesquisa. Leia por curiosidade; não são entregáveis, e tentar construí-los é o modo mais eficiente de perder o semestre.

G.3. O Fragmented Learning sobre as sete camadas

Esta é a figura mais importante do anexo, e a que a conclusão da AV2 precisa retomar. Ela diz exatamente o que o projeto de vocês toma e o que ele concede.



Figura 13 — o que o projeto da disciplina toma em cada camada, e onde declara dependência

Duas leituras importam. A primeira é para não subestimar: um nó que roda inferência local, aprende padrões próprios e decide sem nuvem toma a camada 3 por inteiro e parte da camada 4, que é exatamente onde o paper localiza a soberania alcançável. A segunda é para não exagerar: nada no projeto toca as camadas 1 e 2, nada constrói regime transfronteiriço, e a confiança institucional da camada 6 só aparece como substrato técnico, não como confiança de fato — porque confiança institucional acumula em escala de década, e não de semestre.

G.4. A cascata de resolução

O paper organiza a resolução de uma consulta como uma cascata que tenta o caminho mais local primeiro. O Fragmented Learning implementa a mesma cascata com um degrau deliberadamente removido.



Figura 14 — a cascata do paper e a cascata do Fragmented Learning, com o degrau de internet removido por decisão de projeto

A remoção do terceiro degrau é a decisão de soberania do projeto, e vocês devem sabê-la defender. O paper mantém o degrau de internet porque trata de um sistema de conhecimento geral, em que o novo genuíno precisa vir de algum lugar; o Fragmented Learning o remove porque o dado que ele processa é rastro de localização de uma pessoa identificável, e para esse dado a resposta certa à ignorância não é buscar fora, é abster-se.

G.5. O que está fora de escopo

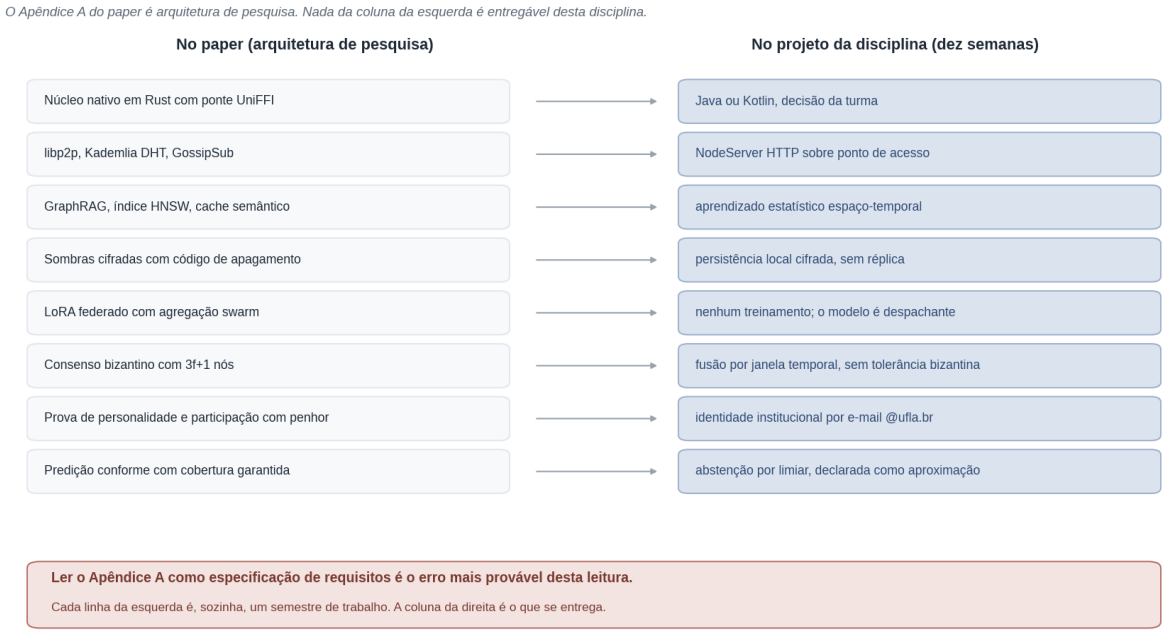


Figura 15 — o que o Apêndice A do paper especifica e o que a disciplina efetivamente constrói

Leia esta figura antes de abrir o Apêndice A

O erro mais provável desta leitura é tratar o Apêndice A como especificação de requisitos. Ele descreve núcleo em Rust, malha libp2p com tabela hash distribuída, grafo de conhecimento com índice vetorial, código de apagamento com prova de replicação, adaptadores federados com privacidade diferencial, consenso bizantino e prova de personalidade. Cada uma dessas linhas é, sozinha, um semestre. A coluna da direita da figura é o que se entrega.

G.6. Como citar

Quando o relatório ou a apresentação usarem o paper, cite-o como de Miranda Neto (2026b), com a seção. Referência completa na seção 16. Duas exigências de honestidade intelectual, ambas verificadas na correção:

- As figuras 7, 8, 9, 14 e 16 do paper são simulações com parâmetros sintéticos, e o próprio texto diz isso. Citá-las como resultado empírico é erro metodológico, e é justamente o erro que a AV2 pune quando exige dispersão nas suas próprias medições. Se usá-las, diga que são ilustrativas.
- O coeficiente K7 é, no arcabouço de origem, construto declaradamente tentativo, com limiares ilustrativos. Não o trate como grandeza medida.

G.7. A contraleitura, que é obrigatória

Ler uma tese sem ler quem discorda dela é doutrinação, não formação. O próprio paper indica os dois contrapontos mais fortes, e a turma deve conhecê-los:

Kokotajlo et al. (2025)	**O cenário AI 2027 projeta concentração em vez de difusão, com uma explosão de capacidade conduzida por pesquisa automatizada. Se ele estiver certo, a curva de aproximação do paper vira perseguição a um alvo que acelera. O paper responde a isso na seção 15, e a resposta é honesta: a soberania de gênese fica definitivamente perdida nesse ramo. Leiam a resposta e julguem se ela basta.**
Epoch AI (2025)	A análise da escala do treinamento descentralizado mostra que os maiores modelos treinados assim usaram cerca de mil vezes menos computação que a fronteira contemporânea. É o dado que mais restringe o otimismo da tese, e o paper o cita contra si mesmo.

G.8. Duas coisas que o paper trouxe para dentro da avaliação

A leitura não é decorativa: ela alterou o projeto em dois pontos concretos.

Abstenção calibrada	**Passou a ser requisito. O RuleEngine ganhou uma quarta saída e o catálogo ganhou a ferramenta declarar_evidencia_insuficiente. A ideia vem da seção 7 e do Apêndice A.7 do paper, onde não saber ainda é tratado como resposta válida e segura, e não como falha. Atenção ao limite: o paper propõe abstenção calibrada por predição conforme, com garantia de cobertura; o que vocês implementam é abstenção por limiar. São coisas diferentes, e o relatório deve dizer isso.**
Cache semântico	Entrou como extensão opcional de nível Excelente na AV2, a partir da seção 12.2. A ligação com o que vocês já medem é direta: uma consulta servida do cache não consome prefill algum, e prefill é a fração serial que a Lei de Amdahl vai expor na semana 10. Quem implementar deve medir o efeito, não apenas descrevê-lo.

A pergunta que fecha o relatório da AV2
Quais camadas o nosso sistema efetivamente tomou, e em quais permanecemos dependentes? Responder isso com a Figura 13 na mão, sem | exagerar o que foi conquistado e sem esconder o que foi concedido, é o fecho que a disciplina espera. É também, em escala de turma, exatamente a pergunta que o paper faz em escala de instituição.

Anexo H — Funcionalidades opcionais de extensão

Este anexo lista funcionalidades opcionais.

Aprendizado persistente e configuração. LearningModel acumula e prevê a rotina: segmenta visitas e registra lugares, transições e tempos; devolve o destino provável, a probabilidade e o tempo estimado; combina outro modelo de forma federada.

Armazenamento local e cifra em repouso. banco SQLite local; LearningStore.save(userId, model) e load(userId, radiusM) gravam e recarregam o aprendizado, cifrando as coordenadas; ConfigStore guarda a configuração por usuário; UserStore administra os perfis do aparelho.

Segundo plano e bateria. PowerPolicy.decide(...) define a cadência de amostragem de forma adaptativa, para monitorar de forma contínua sem esgotar a bateria; MonitoringScheduler.ensure(context) mantém o serviço vivo através do Doze com uma tarefa periódica do WorkManager; MonitoringWorker é a rede de segurança periódica; BootReceiver rearma o monitoramento após reiniciar o aparelho.